# 机器人SLAM与Nav应用开发 实验指导书（合并版）

> 课程代码: EXP004
> 总学时: 48
> 实验数: 7
> 适用对象: 人工智能/机器人工程/智能科学与技术 本科高年级（初学者友好）
> 制定日期: 2026-08-14
> 模式: human_waiver（源码位于 ROS2 工作空间 src/，未复制到 source-codes/EXP004/）

## 目录


1. 实验1：SLAM建图原理与URDF模型加载实验
2. 实验2：基于SLAM_Toolbox的二维激光建图实验
3. 实验3：基于Nav2的机器人自主导航实验
4. 实验4：基于AMCL的自适应蒙特卡罗定位实验
5. 实验5：局部路径规划与动态避障实验
6. 实验6：基于激光雷达的单点与多点导航避障实验
7. 实验7：基于RTAB-Map的三维视觉SLAM导航实验


## 实验总览

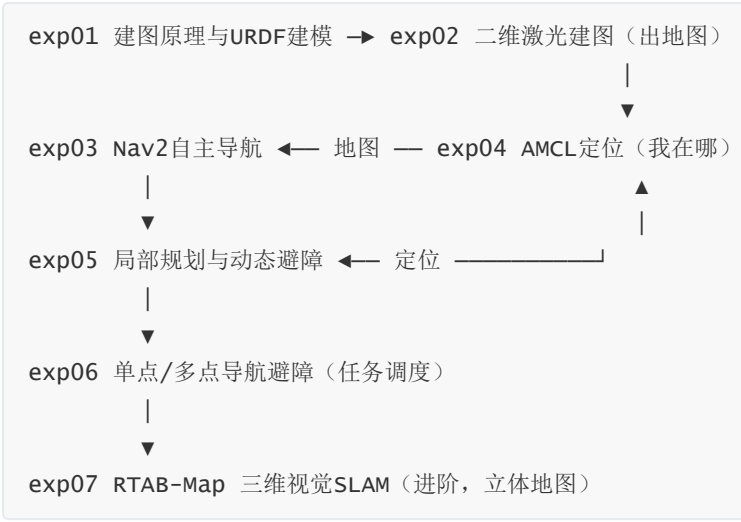
| 编号 | 名称 | 学时 | 类型 | 难度 | 源码路径 |
|---|---|---|---|---|---|
| exp01 | SLAM建图原理与URDF模型加载实验 | 4h | 验证性 | ⭐ | `ros2_ws/src/xflz_eai_car1/simulations/car1_description/` + `slam/car1_slam/` |
| exp02 | 基于SLAM_Toolbox的二维激光建图实验 | 4h | 设计性 | ⭐⭐ | `ros2_ws/src/xflz_eai_car1/slam/car1_slam/` |
| exp03 | 基于Nav2的机器人自主导航实验 | 8h | 综合性 | ⭐⭐⭐ | `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/` |
| exp04 | 基于AMCL的自适应蒙特卡罗定位实验 | 8h | 综合性 | ⭐⭐⭐ | `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/` |
| exp05 | 局部路径规划与动态避障实验 | 8h | 综合性 | ⭐⭐⭐ | `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/` |
| exp06 | 基于激光雷达的单点与多点导航避障实验 | 8h | 综合性 | ⭐⭐⭐ | `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/` |
| exp07 | 基于RTAB-Map的三维视觉SLAM导航实验 | 8h | 综合性 | ⭐⭐⭐ | 无资源（原理+演示型，human_waiver） |

**学时合计**: 4+4+8+8+8+8+8 = **48h**

**类型分布**: 验证性 1 个 + 设计性 1 个 + 综合性 5 个

**难度分布**: ⭐ 1 个 + ⭐⭐ 1 个 + ⭐⭐⭐ 5 个

# 跨实验知识脉络




> 核心链路一句话：**先建模（exp01）→ 建图（exp02）→ 定位（exp04）→ 导航（exp03/05/06）**，exp07 为三维进阶。

# 实验快速验证命令（完整闭环）

```
# 一键建图（bringup + 雷达 + slam_toolbox + RViz）
ros2 launch car1_slam slam.launch.py

# 遥控建图（另开终端，真终端运行）
ros2 run car1_slam slam_teleop

# 保存地图
ros2 run car1_slam map_save map_01

# 一键导航（加载地图 + AMCL 定位 + Nav2 五大服务器 + RViz）
ros2 launch car1_navigation navigation.launch.py map:=map_01
```

# 实验1：SLAM建图原理与URDF模型加载实验（4h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验源码位于 `ros2_ws/src/xflz_eai_car1/simulations/car1_description/`（URDF）与 `ros2_ws/src/xflz_eai_car1/slam/car1_slam/`，未复制到 `source-codes/EXP004/exp01/`。已通过 human_waiver 模式放行，内容基于 `car1.urdf`、`display.launch.py`、`slam_toobox_tutorial.md` 等真实源码撰写。

**一句话读懂**：这节课不写一行复杂代码，目标就两件事——① 用"蒙眼逛新家"的比喻听懂 SLAM 到底在干什么；② 用 URDF + RViz 把 car1 机器人"搬"到电脑屏幕上，看看它长什么样、坐标怎么连。

## 1.1 关联知识点

- 1.4.1 SLAM概念 > 概念 > SLAM = 同时定位与建图，"鸡生蛋蛋生鸡"问题
- 1.4.2 建图三要素 > 原理 > 感知 → 定位 → 构图，循环往复

- 1.4.3 占据栅格地图 > 概念 > 0 空闲 / 100 占据 / -1 未知
- 1.1.1 URDF概念 > 概念 > `<link>` 身体部件 + `<joint>` 关节连接
- 1.1.2 joint类型 > 原理 > fixed 固定 / continuous 车轮 / revolute 舵机
- 1.1.3 TF树 > 概念 > URDF 自动生成，根节点 base_footprint
- 1.2.1 Xacro宏 > 概念 > 宏语言复用部件模板
- 1.3.1 car1三部分 > 概念 > 主体 + 机械臂 + 爪子
- 1.3.2 display三件套 > 流程 > joint_state_publisher_gui + robot_state_publisher + rviz2
- 1.5.1 建图环境 > 流程 > 驱动 + 雷达 + SLAM + RViz
- 1.5.2 好地图标准 > 概念 > 墙单线/尺寸误差<2%/直角≈90°

## 1.2 实验目标

1. 理解 SLAM 的基本概念与建图三要素 — **原理理解**
2. 掌握 URDF 的 link/joint 结构与 Xacro 宏语法 — **流程掌握**
3. 独立完成 car1 模型的 RViz 可视化加载 — **独立完成**
4. 能够判断什么样的地图是"好地图" — **理解**

## 1.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 工作站（Ubuntu 22.04 + ROS2 Humble）— 本实验可**不接真机**，纯软件可视化 |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | ROS2（urdf / xacro / robot_state_publisher / joint_state_publisher_gui / rviz2） |
| 依赖 | `source /opt/ros/humble/setup.bash` + `source ~/ros2_ws/install/setup.bash`；已 `colcon build --packages-select car1_description` |
| 主入口 | `simulations/car1_description/launch/display.launch.py`、`simulations/car1_description/urdf/car1.urdf` |

## 1.4 实验难度

- 难度: ⭐（验证性，入门）
- 重点: SLAM 直觉理解；URDF 的 link/joint 概念；RViz 可视化
- 难点: 无（本实验为全课程铺垫）

## 1.5 实验现象

**快速验证流程**（帮助学习者快速看到实验效果）：

1. 打开终端：`ros2 launch car1_description display.launch.py`
2. RViz 窗口打开，左侧显示 car1 机器人 3D 模型（底盘 + 4 个麦轮 + 机械臂 + 爪子）

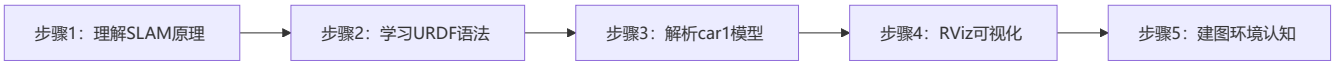
3. 左侧面板打开 "JointStatePublisher" 面板，拖动机械臂关节滑块，模型实时跟随
4. 打开 TF 显示（Add → TF），看到 base_footprint → base_link → ... → laser_link 的坐标轴树

**预期效果描述**：

- RViz 中机器人以网格/实体显示，底盘 4 麦轮可绕轮轴旋转（continuous 关节）
- 机械臂 5+ 关节滑块可拖动，爪子可张合（revolute 关节）
- TF 树显示红色 X / 绿色 Y / 蓝色 Z 坐标轴，从 base_footprint 逐级连接
- 关闭 display.launch 后无任何硬件动作（本实验纯仿真观察）

# 1.6 实验步骤

## 整体流程图




---

## 步骤1：理解 SLAM 原理（30min）

**知识点讲解**：

**SLAM 概念（1.4.1）**

- **原理**：SLAM（Simultaneous Localization and Mapping）中文叫"同时定位与建图"。核心矛盾是：**定位需要地图，建图需要知道自己位置**——像"鸡生蛋、蛋生鸡"。解决思路是"一边猜一边画，互相校正"：先用粗略位置画个粗略地图，再用这个地图修正位置，越修越准。
- **白话类比**：蒙上眼睛走进一个新房间，你只能靠手摸墙（雷达扫描）和数自己走了几步（里程计）。你要么先大概知道自己在哪才能画地图，要么先有地图才能定位——解决办法是"边摸边画边猜"：摸到墙就记住"这里有一堵墙"，同时根据墙的位置猜"我可能在这附近"。SLAM 就是这个过程的高速版。
- **源码对应**：本课程使用 `slam/car1_slam/launch/slam.launch.py` 启动的 slam_toolbox 就是干这个活的（详见实验 2）。

**建图三要素（1.4.2）**

- **原理**：一次建图循环包括三步：① **感知**——激光雷达发出激光，扫到墙反射回来，得到一圈"到障碍的距离"（/scan）；② **定位**——结合上一帧的位置和里程计，猜"我现在在哪"；③ **构图**——把这一圈的墙写进地图。机器人每走一小步，就重复一次这三步。
- **源码对应**：感知来自 `peripherals/car1_peripherals/launch/lidar.launch.py` 启动的 COIN-D6 雷达（发布 `/scan`，frame 为 `laser_link`，10Hz）。

**占据栅格地图（1.4.3）**

- **原理**：电脑里的地图是一张张"小方格"组成的，叫占据栅格地图（OccupancyGrid）。每个格子有三个取值：**-1 未知**（没扫到过）、**0 空闲**（确定没东西）、**100 占据**（确定有墙）。分辨率 `resolution: 0.05` 表示每格边长 5cm。
- **白话类比**：就像用方格纸画房间平面图——纸上的格子要么空着（空闲），要么涂黑（墙），要么没画（未知）。

**操作步骤**：

1. 阅读课程知识图谱 `EXP004_knowledge_graph.md` 第 1 章，理解四个核心概念。
2. 用自己的话说一遍："SLAM 为什么是鸡生蛋蛋生鸡？解决办法是什么？"写在实验笔记中。
3. 查一下雷达话题：`ros2 topic hz /scan`（有真机时执行，观察 ~10Hz）。

**易错点**：

- ⚠️ 把"定位"和"建图"当成两个独立问题 → **解决**：记住它们是互相依赖的循环，这是理解 SLAM 一切设计（回环检测、粒子滤波）的钥匙。

---

## 步骤2：学习 URDF 语法（45min）

**知识点讲解**：

**URDF 结构（1.1.1 + 1.1.2）**

- **原理**：URDF（Unified Robot Description Format）用 XML 描述机器人。两个核心标签：`<link>` 表示机器人的一个"身体部件"（比如底盘、大腿、爪子），定义它的尺寸、颜色、惯性；`<joint>` 表示两个 link 怎么连接，决定它们之间能否运动：
  - `type="fixed"`：焊死，不能动（如 base_footprint → base_link）
  - `type="continuous"`：连续旋转，没有角度限制（如车轮）
  - `type="revolute"`：有限旋转，有角度上下限（如机械臂关节）
- **源码对应**：`simulations/car1_description/urdf/car1.urdf` L7-11 展示了 fixed 关节的例子：

```
<link name="base_footprint" />
<joint name="base_joint" type="fixed">
  <origin xyz="0 0 0.029" rpy="0 0 0" />
  <parent link="base_footprint" />
  <child link="base_link" />
</joint>
```

**TF 树（1.1.3）**

- **原理**：机器人所有坐标系的关系组成一棵"树"，叫 TF 树。URDF 加载后会自动生成：`base_footprint`（根）→ `base_link` → `laser_link`（雷达）/ `imu_link`（IMU）/各机械臂关节。SLAM 和导航都靠查这棵树做坐标换算。
- **源码对应**：`bringup/launch/bringup.launch.py` L32-38 用 robot_state_publisher 加载 URDF 并发布静态 TF。

**Xacro 宏（1.2.1）**

- **原理**：Xacro 是 URDF 的"宏"语言。如果一辆车有 4 个相同的轮子，不用复制粘贴 4 遍，用 `<xacro:macro name="wheel">...</xacro:macro>` 定义一次，然后调用 4 次即可。改一个参数（如轮径）所有轮子一起变。
- **源码对应**：`simulations/car1_description/gazebo/car1.gazebo.xacro` 用 xacro 扩展了 Gazebo 仿真所需的物理属性。

**操作步骤**：

1. 打开 `car1.urdf`，数一数有多少个 `<link>` 和 `<joint>`（`grep -c "<link" car1.urdf`）。

2. 找 3 种关节类型各一个例子，记录它们的 parent/child。
3. 阅读 `display.launch.py`，说出它启动了哪三个节点（答案：joint_state_publisher_gui、robot_state_publisher、rviz2）。

**易错点**：

- ⚠️ 混淆 link 和 joint → **解决**：link 是"身体"，joint 是"关节/螺丝"，joint 永远连接两个 link（parent 和 child）。
- ⚠️ 以为 TF 是手动画的 → **解决**：URDF 加载后 TF 树自动生成，机器人坐标关系不用自己写代码。

---

## 步骤3：解析 car1 模型结构（45min）

**知识点讲解**：

**car1 三大部分（1.3.1）**

- **原理**：`car1.urdf` 描述的 car1 机器人由三部分拼成：
  1. **主体**：`base_footprint` → `base_link`（底盘）+ 4 个麦轮（continuous 关节，麦轮 = 全向轮，能横着走）
  2. **机械臂**：`arm_link1` ~ `arm_link6` 的串联链（revolute 关节，每个关节一个舵机）
  3. **爪子**：末端夹爪（gripper），用来抓取物体
- **源码对应**：`car1.urdf` 中 base_joint（fixed）连接底盘；4 个轮子关节为 continuous；机械臂关节为 revolute。

**操作步骤**：

1. 打开 `car1.urdf`，从上到下浏览，圈出三大部分的 link/joint 名称。
2. 对照下表记录：

| 部分 | 代表 link | 关节类型 | 说明 |
| --- | --- | --- | --- |
| 主体 | base_link | fixed/continuous | 底盘 + 4 麦轮 |
| 机械臂 | arm_link1~6 | revolute | 5 自由度舵机臂 |
| 爪子 | gripper | revolute | 末端夹爪 |

3. 思考题（记录在笔记）：为什么底盘要单独用 base_footprint 而不是直接 base_link 当根？（提示：base_footprint 表示"投影在地面的参考点"，方便 2D 导航）

**易错点**：

- ⚠️ 找不到机械臂关节 → **解决**：搜索 `type="revolute"` 关键字即可定位。
- ⚠️ 混淆"自由度"概念 → **解决**：revolute 关节数量 = 自由度，car1 机械臂 5 个旋转关节 = 5 自由度。

---

## 步骤4：RViz 可视化验证（60min）

**知识点讲解**：

**display.launch.py 三件套（1.3.2）**

- **原理**：`display.launch.py` 一行命令启动三件事：
    1. **joint_state_publisher_gui**：启动一个 GUI 面板，每个关节一个滑块，拖动滑块就发布关节角度（/joint_states）
    2. **robot_state_publisher**：读 URDF + 关节角度，实时计算每个 link 的位姿，发布 TF
    3. **rviz2**：把 TF 画成 3D 模型显示在屏幕上
- **源码对应**：`simulations/car1_description/launch/display.launch.py` L17-36 依次启动三个节点。

**操作步骤**：

1. 启动可视化：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch car1_description display.launch.py
```

2. RViz 出现机器人模型后，操作：
    - 左侧面板展开 "JointStatePublisher"（或 Add 面板），拖动 arm_joint1~5 滑块 → 机械臂跟随
    - Add → By topic → /tf → 显示 TF 坐标轴
    - 点击 "Interact" 工具 → 拖拽模型任意部位观察
3. 验证 TF 树：

```
ros2 run tf2_tools view_frames   # 生成 frames.pdf，用浏览器打开
```

   观察 base_footprint 是否为根，laser_link / imu_link 是否挂在 base_link 下。
4. 思考：如果把轮子关节滑块拖到极限，模型会怎样？为什么 continuous 关节没有角度限制？

**易错点**：

- ⚠️ RViz 只显示坐标轴看不到模型 → **解决**：左侧 Add → RobotModel → 选择 /robot_description 话题。
- ⚠️ 关节滑块拖不动 → **解决**：确认 joint_state_publisher_gui 已启动（display.launch 已包含），且 RViz 勾选了对应 Display。
- ⚠️ `ros2 launch` 报找不到包 → **解决**：先 `source ~/ros2_ws/install/setup.bash`，确认已 build car1_description。

---

## 步骤5：建图环境认知（45min）

**知识点讲解**：

**建图环境构成（1.5.1）**

- **原理**：真正建图时，系统由四层组成（本实验只认识，不启动）：
    1. **驱动层** `bringup.launch.py`：底盘驱动 + 机器人状态发布 + EKF 里程计（把轮速 + IMU 融合成 odom TF）
    2. **雷达层** `lidar.launch.py`：发布 /scan
    3. **SLAM 算法层**：slam_toolbox（实验 2 详解）

4. **可视化层**：RViz

- **源码对应**：`slam/car1_slam/launch/slam.launch.py` L36-59 正是这种分层结构。

**好地图标准（1.5.2）**

- **原理**：一张"好地图"长这样：墙是**单线**（没有重影）、尺寸误差 < 2%、墙角是**直角**、绕一圈回到起点时位置误差小（闭环成功）。这些指标在实验 2 会实际测量。
- **白话类比**：好地图 = 清晰、没有"重影"、比例尺准的房间平面图。

**操作步骤**：

1. 打开 `slam/car1_slam/launch/slam.launch.py`，对照四层结构找对应代码（bringup 包含 / lidar 包含 / TimerAction 10s slam_toolbox / rviz 节点）。
2. 打开 `slam/car1_slam/config/slam.yaml`，找到 `mode: mapping`、`resolution: 0.05`、`do_loop_closing: true` 三个参数。
3. 阅读 `slam/doc/slam_toobox_tutorial.md` 第 8 节"检验标准"，记录 4 个量化指标。
4. 在实验笔记中绘制"建图四层结构"图。

**易错点**：

- ⚠️ 把"建图"和"导航"混为一谈 → **解决**：建图（实验 2）产出地图，导航（实验 3-6）消费地图，先有图才能导。
- ⚠️ 以为分辨率越小越好 → **解决**：0.05（5cm）已足够室内导航；再小地图文件巨大、算力吃紧、收益有限。

## 1.7 注意事项

- ⚠️ 本实验**不需要真机**，纯软件即可完成（有真机可顺带验证 /scan 话题）。
- ⚠️ 若使用真机，display.launch 只加载模型不动电机，安全无风险。
- ⚠️ view_frames 生成的 frames.pdf 可用 `firefox frames.pdf` 或 VS Code 打开。

## 1.8 实验总结

**完成成果**：

- 理解 SLAM 的"鸡生蛋蛋生鸡"本质与建图三要素
- 掌握 URDF 的 link/joint/fixed/continuous/revolute 语法
- 成功在 RViz 中加载 car1 模型并拖动关节滑块
- 认识建图四层结构并记录好地图标准

**技能收获**：

- 技能1：SLAM 直觉理解（类比法）
- 技能2：URDF 模型阅读能力
- 技能3：RViz + TF 树可视化操作

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| RViz 无模型 | 未选择 RobotModel Display | Add → RobotModel |
| 关节拖不动 | joint_state_publisher_gui 未启动 | display.launch 已包含，检查终端输出 |
| launch 报包找不到 | 未 source install | `source ~/ros2_ws/install/setup.bash` |

## 1.9 实验报告要求

**思考题**:

1. SLAM 为什么是"鸡生蛋蛋生鸡"问题？解决这个矛盾的核心思路是什么？
2. `<joint type>` 的三种类型各适合什么场景？为什么车轮用 continuous、机械臂用 revolute？
3. 占据栅格地图中 -1/0/100 分别代表什么？分辨率 0.05 表示什么？
4. 为什么导航前必须先建图？地图质量差会对导航产生什么影响？

**评分标准**：SLAM 原理理解 20 分 + URDF 语法 20 分 + car1 模型解析 20 分 + RViz 可视化操作 25 分 + 报告与思考题 15 分 = 100 分。

---

# 实验2：基于SLAM_Toolbox的二维激光建图实验（4h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验源码位于 `ros2_ws/src/xflz_eai_car1/slam/car1_slam/`，未复制到 `source-codes/EXP004/exp02/`。已通过 human_waiver 放行，内容基于 `launch/slam.launch.py`、`config/slam.yaml`、`car1_slam/map_save.py`、`car1_slam/slam_teleop.py`、`maps/slam_map.yaml`、`doc/slam_toobox_tutorial.md` 等真实源码撰写。

**一句话读懂**：这节课是整门课的第一个"高光时刻"——你将用键盘遥控小车在房间里逛一圈，然后得到一张跟户型图一样的二维地图。全程只做三件事：**启动建图 → 遥控逛圈 → 保存地图。**

## 2.1 关联知识点

- 2.1.1 slam_toolbox框架 > 概念 > Karto 图优化 SLAM：前端 Scan Match + 位姿图 + 后端 Ceres
- 2.1.2 前端ScanMatch > 原理 > 在里程计预测位姿附近搜候选，取匹配得分最高
- 2.1.3 后端图优化 > 原理 > 节点=关键帧，边=约束，Ceres 最小化残差
- 2.1.4 回环检测 > 流程 > 回到已访问区域 → 一次大修正
- 2.2.1 resolution > 概念 > 0.05 = 每格 5cm
- 2.2.2 关键帧阈值 > 流程 > minimum_travel_distance/heading 控制更新频率
- 2.2.3 mode/loop_closing > 流程 > mapping 模式 + 开启回环
- 2.3.1 slam.launch分层 > 流程 > bringup → 雷达 → [10s] slam_toolbox → RViz
- 2.3.2 键盘遥控 > 流程 > slam_teleop：w/s/a/d/空格
- 2.3.3 建图手法 > 原理 > 慢速、闭环、拐角减速
- 2.4.1 保存地图 > 流程 > map_saver_cli 产出 pgm+yaml
- 2.4.2 pgm/yaml格式 > 概念 > 255 空闲 / 0 占据 / 205 未知
- 2.5.1 验收四指标 > 流程 > 墙厚/尺寸/直角/闭环

## 2.2 实验目标

1. 理解 slam_toolbox 的图优化框架（前端匹配 + 后端优化 + 回环）— **原理理解**
2. 掌握 slam.yaml 关键参数（分辨率/关键帧阈值/回环开关）的配置方法 — **流程掌握**
3. 独立完成"启动建图 → 键盘遥控 → 保存地图 → 质量评估"全流程 — **独立完成**
4. 能够分析地图重影/旋转等常见问题并给出修正方向 — **迁移应用**

## 2.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 具身智能实验箱B2 / 小车C1 + COIN-D6 激光雷达（/dev/ttyUSB1） |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | slam_toolbox、robot_localization、nav2_map_server（保存地图用） |
| 依赖 | `source /opt/ros/humble/setup.bash` + `source ~/ros2_ws/install/setup.bash`；`colcon build --packages-select car1_slam car1_bringup car1_peripherals` |
| 主入口 | `slam/car1_slam/launch/slam.launch.py`（建图）、`car1_slam/slam_teleop.py`（遥控）、`car1_slam/map_save.py`（保存） |

## 2.4 实验难度

- 难度: ⭐⭐（设计性）
- 重点: 建图全流程；关键参数对地图质量的影响
- 难点: 参数与建图手法的配合（阈值大小、遥控速度）

## 2.5 实验现象

**快速验证流程**（建议在 2m×2m 以上空旷室内）：

1. 启动建图：`ros2 launch car1_slam slam.launch.py`（自动带起底盘 + 雷达 + slam_toolbox + RViz）
2. 新终端启动遥控：`ros2 run car1_slam slam_teleop`（或 `python3 src/xflz_eai_car1/slam/car1_slam/car1_slam/slam_teleop.py`）
3. 用 w/a/s/d 缓慢遥控小车沿墙走一圈，回到起点
4. RViz 中看到地图随移动逐渐"长"出来：墙成单线、房间轮廓闭合
5. 保存地图：`ros2 run car1_slam map_save map_01`，在 `slam/car1_slam/maps/` 下看到 map_01.pgm + map_01.yaml

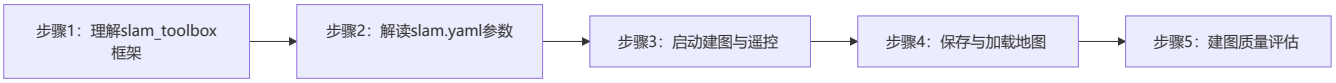
**预期效果描述**:

- RViz 左侧 Map 显示实时占据栅格（白色=空闲、黑色=墙、灰色=未知）
- 小车位置显示绿色箭头，跟随地图一起走
- 地图质量：墙为单线、房间角度接近直角、绕一圈后回到起点墙线不错开

- 保存的 map_01.pgm 用图片查看器打开就是一张清晰的"户型图"

## 2.6 实验步骤

### 整体流程图




---

### 步骤1：理解 slam_toolbox 框架（40min）

**知识点讲解**：

**图优化 SLAM（2.1.1 + 2.1.2 + 2.1.3）**

- **原理**：slam_toolbox 把建图拆成"前端"和"后端"两部分：
  - **前端（Scan Matching）**：每来一帧激光扫描，都在"里程计预测的位置附近"（一个搜索窗口）试着把扫描往各个方向放一放，看哪个位置与已有地图最吻合，就认为机器人在这里。对应参数：`correlation_search_space_dimension: 0.5`（搜索窗 ±0.5m）、`correlation_search_space_resolution: 0.01`（步进 1cm）。
  - **位姿图（Pose Graph）**：把每个"关键帧"（关键位置的扫描+位姿）记成一个**节点**，相邻关键帧之间的相对运动记成**边**（约束）。
  - **后端（图优化）**：攒了一批节点和边之后，用 Ceres 求解器（`ceres_trust_strategy: LEVENBERG_MARQUARDT`）把所有节点的位置重新调一调，让所有约束都尽量满足——把累积的小误差"摊平"。
- **白话类比**：前端=每走一步都"对对齐"（局部对正）；后端=走累了回头"重新排排站"（全局优化）；回环=绕了一圈发现"这墙我画过"（见 2.1.4）。
- **源码对应**：`config/slam.yaml` L5-10 是后端求解器配置；L48-52 是前端搜索窗口；L30-43 是关键帧与 scan 接收配置。

**回环检测（2.1.4）**

- **原理**：机器人走了一圈回到出发点时，系统在**历史关键帧**里搜索"有没有和现在这帧长得像的"（分支定界搜索，`loop_search_space_dimension: 8.0`）。找到了（匹配响应超过阈值，`loop_match_minimum_response_fine: 0.45`）就建立一条"回环约束"，然后触发一次全局优化——这一次优化能把整条轨迹的累积漂移一次性抹掉。**所以建图一定要走闭环路径。**
- **源码对应**：`slam.yaml` L54-64 回环检测参数（`do_loop_closing: true`）。

**操作步骤**：

1. 打开 `slam.yaml`，分别找到：前端搜索窗口、后端求解器、回环检测、关键帧阈值 四组参数，抄到笔记。
2. 画出 slam_toolbox 数据流图：/scan + odom TF → 前端匹配 → 位姿图 → 后端优化 → /map。
3. 用一句话向同桌解释：为什么"绕一圈回到起点"能让地图突然变准？

**易错点**：

- ⚠️ 以为 SLAM 只靠雷达 → **解决**：还要吃里程计 TF（odom→base_footprint），由 bringup 的 EKF 提供；雷达和里程计都准，地图才准。

- ⚠️ 回环检测不是必然触发 → **解决**：触发需要"现在这帧"和"历史某帧"足够像，路径重叠太少就检测不到。

---

## 步骤2：解读 slam.yaml 关键参数（40min）

**知识点讲解**：

**分辨率（2.2.1）**

- **原理**：`resolution: 0.05` 表示地图每格 5cm。格子越小地图越精细（细节多），但地图文件越大、每次更新计算越重。室内导航 0.05 是黄金值。
- **源码对应**：`slam.yaml` L23。

**关键帧阈值（2.2.2）**

- **原理**：不是每帧扫描都存成关键帧，而是"走得够远了才存"。`minimum_travel_distance: 0.01`（走了 1cm）或 `minimum_travel_heading: 0.02`（转了 0.02rad）才触发新关键帧。**阈值太大**→关键帧太少→地图粗糙、回环失败；**阈值太小**→关键帧太多→算力浪费。
- **源码对应**：`slam.yaml` L40-41。文档中特别注释：没有这两个阈值 slam_toolbox 会退回 0.5m/0.5rad 默认值，地图只有大动作才更新。

**模式与回环开关（2.2.3）**

- **原理**：`mode: mapping` 表示建图模式（导航定位时改为 localization）；`map_update_interval: 0.1` 表示地图每 0.1s 刷新一次；`do_loop_closing: true` 开启回环检测。
- **源码对应**：`slam.yaml` L19、L22、L58。

**操作步骤**：

1. 把下面参数抄入笔记并标注含义：

| 参数 | 值 | 含义 |
| --- | --- | --- |
| resolution | 0.05 | 每格 5cm |
| mode | mapping | 建图模式 |
| map_update_interval | 0.1 | 地图刷新周期 0.1s |
| minimum_travel_distance | 0.01 | 位移 1cm 触发新关键帧 |
| minimum_travel_heading | 0.02 | 转角 0.02rad 触发新关键帧 |
| do_loop_closing | true | 开启回环检测 |
| max_laser_range | 12.0 | 激光截断到 12m |

2. 思考（记录）：若把 minimum_travel_distance 改成 0.5，建图会怎样？（提示：地图只在走够 0.5m 才更新，细节全丢）

**易错点**：

- ⚠️ 改参数后不生效 → **解决**：参数在节点启动时读取，改 slam.yaml 后**重启 slam.launch**；不要在运行时 `ros2 param set` 期望地图立刻变。

---

## 步骤3：启动建图与遥控（60min）

**知识点讲解**：

**slam.launch.py 分层启动（2.3.1）**

- **原理**：一行命令按顺序拉起：① `bringup.launch.py`（底盘驱动 + robot_state_publisher + EKF 里程计）→ ② `lidar.launch.py`（COIN-D6 雷达 /scan）→ ③ **延迟 10s** 才启动 slam_toolbox（等 odom/TF/scan 稳定，避免刚上电数据抖动）→ ④ RViz。
- **源码对应**：`launch/slam.launch.py` L36-59：

```
# 3. slam_toolbox, delayed 10s to let odom / TF / scan stabilize.
slam_node = TimerAction(
    period=10.0,
    actions=[Node(package='slam_toolbox', executable='sync_slam_toolbox_node',
                  name='slam_toolbox', parameters=[slam_config], output='screen')])
```

**键盘遥控（2.3.2）**

- **原理**：`slam_teleop.py` 把键盘按键转成 `/cmd_vel` 速度话题：w=前进 0.2m/s、s=后退、a=左转 0.5rad/s、d=右转、空格=停止。它用 termios 读取原始按键，**必须在终端里直接运行**（不能在后台）。
- **源码对应**：`car1_slam/slam_teleop.py` L16-34（按键映射）、L79-99（发布 Twist）。

**建图手法（2.3.3）**

- **原理**：地图质量 = 算法 × **手法**。规范：线速度 ≤0.3m/s、角速度 ≤1.0rad/s（太快会运动畸变）、拐角处减速、**尽量走闭环**、避免长走廊无特征区域快速直行。
- **白话类比**：就像用手机全景拍照——转太快、走得晃，拍出来就是糊的、断的。

**操作步骤**：

1. 终端 1 启动建图：

   ```
   source /opt/ros/humble/setup.bash && source ~/ros2_ws/install/setup.bash
   ros2 launch car1_slam slam.launch.py
   ```

   观察日志：先看到 bringup 各节点，约 10s 后出现 `slam_toolbox` 启动日志（`Registering sensor`、`SLAM ... started`）。

2. 终端 2 启动遥控（**必须真终端**，非 IDE 内嵌）：

   ```
   ros2 run car1_slam slam_teleop
   ```

   或源码直跑：`python3 ~/ros2_ws/src/xflz_eai_car1/slam/car1_slam/car1_slam/slam_teleop.py`

3. RViz 确认：Map 显示、LaserScan 显示（Reliability 设为 Best Effort，见易错点）。

4. 遥控小车**低速**沿房间四周走一圈，回到起点。中途在墙角停留 1-2s 让扫描充分重叠。

5. 监控数据：

   ```
   ros2 topic hz /scan          # ~10Hz
   ros2 run tf2_ros tf2_echo odom base_footprint   # 观察里程计
   ```

**易错点**：

- ⚠️ RViz 不显示激光点 → **解决**：LaserScan Display 的 Reliability Policy 改为 **Best Effort**（雷达 QoS 是 Best Effort）。
- ⚠️ 遥控无反应 → **解决**：确认 slam_teleop 在真终端运行；先按一下任意方向键激活，再操作。
- ⚠️ 启动 10s 内地图不动 → **解决**：正常！slam_toolbox 延迟 10s 启动，等日志出现再开始动。
- ⚠️ 走太快地图扭曲 → **解决**：降速，角速度控制在 1.0rad/s 以内（遥控 a/d 自带 0.5rad/s 很安全）。

---

## 步骤4：保存与加载地图（40min）

**知识点讲解**：

**保存地图（2.4.1）**

- **原理**：slam_toolbox 建完图后，地图在内存里（话题 /map）。`map_save.py` 节点本质是调用 `nav2_map_server` 的 `map_saver_cli` 把 /map 写成文件。产出两个文件：`.pgm`（灰度位图）+ `.yaml`（元数据）。
- **源码对应**：`car1_slam/map_save.py` L27-31（调用 map_saver_cli）；`maps/slam_map.yaml` 是已保存的示例元数据。

**地图文件格式（2.4.2）**

- **原理**：`.pgm` 中 **255=空闲（白）**、**0=占据（黑）**、**205=未知（灰）**。`.yaml` 记录：image 文件名、resolution（每格米数）、origin（地图左下角在 map 系的位置）、occupied_thresh/free_thresh（灰度转占据/空闲的阈值）。
- **源码对应**：`maps/slam_map.yaml`：

```yaml
image: slam_map.pgm
mode: trinary
resolution: 0.05
origin: [-0.381, -0.482, 0]
occupied_thresh: 0.65
free_thresh: 0.25
```

**操作步骤**：

1. 保存地图：

```
ros2 run car1_slam map_save map_01
# 或手动：
ros2 run nav2_map_server map_saver_cli -f map_01 --ros-args -p
map_subscribe_transient_local:=true
```

2. 确认产物：`ls ~/ros2_ws/src/xflz_eai_car1/slam/car1_slam/maps/map_01.*`
3. 用图片查看器打开 map_01.pgm，对照实际房间检查：墙是否单线、房间形状对不对。
4. 查看 map_01.yaml，记录 resolution 和 origin。
5. 检查地图是否能被导航加载（为实验 3 准备）：`ros2 launch car1_navigation navigation.launch.py map:=map_01`（可先不展开，下一实验详解）。

**易错点**：

- ⚠️ map_save 报"没有话题" → **解决**：保存前确认 slam_toolbox 在运行且 /map 在发布；`ros2 topic hz /map` 检查。
- ⚠️ 地图没建完就保存 → **解决**：保存前先让小车绕完整一圈并回到起点，触发一次回环优化再保存。

---

### 步骤5：建图质量评估（40min）

**知识点讲解**：

**验收四指标（2.5.1）**

- **原理**：用尺子和地图工具实测（文档 `slam_toobox_tutorial.md` 第 8 节）：

| 指标 | 方法 | 合格标准 |
|---|---|---|
| 墙体厚度 | 地图上量同面墙的双线间距 | < 0.1m（接近真实墙厚） |
| 尺寸误差 | 地图测距 vs 卷尺实测 | < 2% |
| 直角度 | 地图量墙角夹角 | 90° ± 2° |
| 闭环误差 | 回起点时位置/航向误差 | 位置 < 0.1m、航向 < 5° |

**操作步骤**：

1. 用卷尺量房间长宽（比如 3m × 2m），再在地图（RViz 的 Measure 工具或图片缩放比）量同一尺寸，算误差百分比。
2. 观察墙线：用图片查看器放大 map_01.pgm，检查是否有"双墙/重影"。
3. 分析报告：若出现重影，从下面"常见问题"中定位原因并写修正方案（不要求本次全部修复）。
4. 结论写入实验报告，附地图截图。

**易错点**：

- ⚠️ 地图旋转/双墙 → **解决**：多为里程计航向不准（IMU scale 错、打滑）。检查 `slam_toobox_tutorial.md` 3.4 节的历史问题表。
- ⚠️ 墙特别厚（>0.1m）→ **解决**：多为遥控太快/转弯太猛，重跑时降速。
- ⚠️ 长走廊地图"发飘" → **解决**：走廊无特征，前端匹配易退化；放慢速度或贴墙走，保持更多重叠。

## 2.7 注意事项

- ⚠️ **建图前检查电量**，避免半路断电地图丢失。
- ⚠️ 遥控全程低速（角速度 ≤1.0rad/s），防止运动畸变与撞墙。
- ⚠️ 建图完成立即保存地图，再停止 launch。
- ⚠️ 停止顺序：先 Ctrl+C 遥控，再 Ctrl+C slam.launch；异常退出可用 `pkill -f slam_teleop` 兜底。
- ⚠️ 地图文件命名别用中文，导航加载路径是硬编码拼接的（`map_name + '.yaml'`）。

## 2.8 实验总结

**完成成果**：

- 理解 slam_toolbox 前端匹配 / 后端图优化 / 回环检测三层机制
- 掌握 slam.yaml 关键参数（分辨率/关键帧阈值/回环）的配置
- 独立完成"启动 → 遥控逛圈 → 保存地图"全流程，得到一张可用地图
- 学会用四指标评估地图质量并定位常见问题

**技能收获**：

- 技能1：一键启动建图系统（slam.launch.py）
- 技能2：键盘遥控与建图手法（慢速+闭环）
- 技能3：地图保存（pgm/yaml）与质量评估

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| RViz 无激光 | QoS 不匹配 | LaserScan Display → Reliability = Best Effort |
| 地图不更新 | 参数未生效/速度太快 | 改 slam.yaml 后重启；降速重跑 |
| 地图双墙 | 里程计航向漂移 | 检查 IMU/EKF（见技术文档 3.4） |
| 回环不触发 | 路径不闭合/阈值严 | 走闭环；调低 response 阈值 |
| 保存失败 | /map 未发布 | 确认 slam 在跑，`ros2 topic hz /map` |

## 2.9 实验报告要求

**思考题**：

1. 前端 Scan Matching、后端图优化、回环检测三者各解决什么问题？为什么说"回环检测"是消除累积漂移的关键？
2. `minimum_travel_distance: 0.01` 的作用是什么？把它改成 0.5 会带来什么后果？
3. 为什么建图要求"慢速 + 闭环路径"？结合激光运动畸变与回环触发条件分析。
4. map_01.yaml 中 `resolution: 0.05` 与 `origin` 的含义是什么？导航加载时如何被使用？

**评分标准**：slam_toolbox 框架理解 20 分 + 参数解读 20 分 + 建图全流程操作 30 分 + 地图保存与质量评估 20 分 + 报告与思考题 10 分 = 100 分。

---

# 实验3：基于Nav2的机器人自主导航实验（8h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验源码位于 `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/`，未复制到 `source-codes/EXP004/exp03/`。已通过 human_waiver 放行，内容基于 `launch/navigation.launch.py`、`config/nav2_params.yaml` 等真实源码撰写。

**一句话读懂**：实验 2 我们给了小车一张地图。这节课让小车"学会看地图自己走路"——你在 RViz 里点一个目标点，它自己规划路线、自己转弯、自己躲开静态障碍，安全到达。这背后是 Nav2 导航栈在工作。

## 3.1 关联知识点

- 3.1.1 五大服务器 > 概念 > planner/controller/behavior/bt_navigator/velocity_smoother
- 3.1.2 Lifecycle > 原理 > 先 inactive 配置再 activate 激活
- 3.2.1 全局规划器 > 概念 > ThetaStarPlanner 在 map 系找无碰撞路径
- 3.2.2 全局代价地图 > 概念 > static + obstacle + inflation 三层
- 3.3.1 局部规划器 > 概念 > DWB FollowPath 在 odom 系跟踪路径
- 3.3.2 局部代价地图 > 概念 > 3m×3m 滚动窗口实时感知
- 3.4.1 行为树 > 原理 > XML 编排导航流程
- 3.4.2 两个导航器 > 概念 > NavigateToPose / NavigateThroughPoses
- 3.5.1 navigation.launch > 流程 > 一键启动导航系统
- 3.5.2 下发目标 > 流程 > RViz Nav2 Goal 或 ros2 action
- 3.5.3 导航三要素 > 原理 > 地图/定位/速度约束

## 3.2 实验目标

1. 理解 Nav2 五大服务器架构与 Lifecycle 生命周期 — **原理理解**
2. 掌握全局/局部代价地图的分层机制与膨胀区作用 — **流程掌握**
3. 独立完成"加载地图 → 设定目标 → 自主到达"的导航闭环 — **独立完成**
4. 能够分析导航失败（撞墙/绕路/超时）原因并修正 — **迁移应用**

## 3.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 具身智能实验箱B2 / 小车C1 + COIN-D6 雷达；需先完成实验 2 得到地图 |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | Nav2（planner/controller/bt_navigator/behavior_server）、slam_toolbox 或已存地图 |
| 依赖 | 地图文件位于 `slam/car1_slam/maps/`；`colcon build --packages-select car1_navigation` |
| 主入口 | `navigation/car1_navigation/launch/navigation.launch.py`、`config/nav2_params.yaml` |

## 3.4 实验难度

- 难度: ⭐ ⭐ ⭐ （综合性）
- 重点: Nav2 架构；导航闭环（地图→定位→规划→执行）
- 难点: 行为树流程理解；代价地图分层与膨胀

## 3.5 实验现象

**快速验证流程**（先在 3m×3m 空旷场地）：

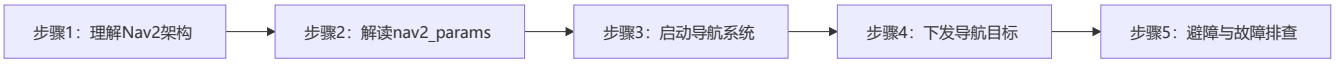
1. 启动导航：`ros2 launch car1_navigation navigation.launch.py map:=slam_map`
2. 观察 RViz：地图加载出来，机器人模型出现在地图内（AMCL 自动初始定位，默认 0,0,0）
3. 点击 RViz 顶部工具栏 **"Nav2 Goal"**，在地图上点一个目标点并拖出朝向
4. 小车自动规划绿色路径 → 沿路径行驶 → 到达后停下，RViz 显示 "Goal reached"
5. 途中放一个箱子（静态障碍）：再次导航时小车路径自动绕开

**预期效果描述**：

- RViz 显示全局路径（绿色曲线）从机器人到目标点，避开了墙和箱子
- 小车实际行驶轨迹与全局路径基本一致，转弯平滑（velocity_smoother 生效）
- 到达后机器人朝向与设定的 yaw 一致；`ros2 action send_goal` 返回值 status 为 4（SUCCEEDED）
- 若在路径上新增静态障碍，Nav2 会**重新规划**绕行

## 3.6 实验步骤

### 整体流程图




---

### 步骤1：理解 Nav2 架构（60min）

**知识点讲解**：

**五大服务器（3.1.1）**

- **原理**：Nav2 把导航拆成五个分工明确的服务器，各管一段：
    1. **planner_server（全局规划）**：在 map 坐标系里，从机器人位置到目标点找一条**全局路径**（避开静态墙）。本项目用 ThetaStarPlanner。
    2. **controller_server（局部规划）**：在 odom 坐标系里，**实时跟踪**全局路径，输出速度指令（cmd_vel），同时躲避动态障碍。本项目用 DWB（FollowPath）。
    3. **bt_navigator（行为树大脑）**：用行为树编排整个流程——"先规划、再跟踪、检查到没到、没到就恢复"。
    4. **behavior_server（行为服务器）**：提供异常恢复动作：spin（原地转）、backup（后退）、drive_on_heading（沿朝向直行）、assisted_teleop（辅助遥控）、wait。
    5. **velocity_smoother（速度平滑器）**：把下发的速度指令"软着陆"，防止急停急启，保护电机。
- **白话类比**：planner 是"看地图设计划线路的导航员"；controller 是"看路况实际开车的司机"；bt_navigator 是"总指挥"；behavior_server 是"故障救援队"；velocity_smoother 是"安全员"。
- **源码对应**：`config/nav2_params.yaml` 中 planner_server（L173-184）、controller_server（L113-172）、bt_navigator（L52-112）、behavior_server（L185-212）、velocity_smoother（L213-227）五段配置。

**Lifecycle 生命周期（3.1.2）**

- **原理**：Nav2 服务器都实现了 Lifecycle 状态机：启动后先 **inactive**（配置好但不出力）→ 收到激活命令变 **active**（开始工作）→ 停止变 **deactivated**。`lifecycle_manager_navigation` 统一管理（autostart: true 自动激活）。
- **源码对应**：`nav2_params.yaml` L326-338 lifecycle_manager_navigation（管理 controller/planner/behavior/bt_navigator/velocity_smoother 五个节点）。

**操作步骤**：

1. 打开 `nav2_params.yaml`，对照五大服务器把每个的 plugin 名抄到笔记。
2. 用 `ros2 node list`（启动后）观察这些节点，并 `ros2 lifecycle get /planner_server` 查看生命周期状态。
3. 画一张"目标点 → bt_navigator → planner → controller → cmd_vel → 底盘"的数据流图。

**易错点**：

- ⚠️ 混淆 planner 和 controller → **解决**：planner 管"走哪条路"（全局，慢），controller 管"怎么走"（局部，快），缺一不可。
- ⚠️ 以为 lifecycle 节点启动即工作 → **解决**：要等 lifecycle_manager 激活（autostart 自动做），激活前节点 inactive。

---

## 步骤2：解读 nav2_params 关键配置（60min）

**知识点讲解**：

**全局代价地图（3.2.2）**

- **原理**：`global_costmap` 在 **map** 坐标系，三层叠加：
  - **static_layer**：读入实验 2 建好的静态地图（`map_subscribe_transient_local: true` 订阅 /map）
  - **obstacle_layer**：实时雷达点云标记动态障碍（scan topic /scan）
  - **inflation_layer**：把障碍**向外"吹"一层膨胀区**（`inflation_radius: 0.18`），代价从障碍处向外衰减，让机器人保持安全距离
- **源码对应**：`nav2_params.yaml` L268-301 global_costmap 三段 plugins。

**全局规划器（3.2.1）**

- **原理**：本项目 `planner_plugins: ["GridBased"]`，plugin 为 `nav2_theta_star_planner/ThetaStarPlanner`（带视线检查的 A* 变体，路径更直）。它在 global_costmap 上搜索从起点到目标的最优无碰撞路径。
- **源码对应**：`nav2_params.yaml` L173-184。

**局部代价地图与局部规划（3.3.1 + 3.3.2）**

- **原理**：`local_costmap` 在 **odom** 坐标系，是跟着机器人走的**滚动窗口**（3m×3m，`rolling_window: true`），只关心身边 3 米内的障碍。局部规划器 `DWB FollowPath`（`dwb_core::DWBLocalPlanner`）在这个小窗口里实时生成短轨迹跟踪全局路径。
- **源码对应**：`nav2_params.yaml` L228-266（local_costmap）、L133-171（FollowPath）。

**操作步骤**：

1. 抄录两个代价地图的关键差异表：

| 项 | global_costmap | local_costmap |
| --- | --- | --- |
| 坐标系 | map | odom |
| 范围 | 全图 | 3m×3m 滚动窗口 |
| 图层 | static+obstacle+inflation | voxel+inflation |
| 更新频率 | 1.0Hz | 5.0Hz |

2. 找出 footprint（机器人轮廓）：`footprint: "[[0.18,0.135],...]"`，理解 0.18/0.135 是半长/半宽。
3. 思考（记录）：inflation_radius 从 0.18 改到 0.5 会怎样？（提示：机器人会把所有障碍当"危险区"绕很远，窄门过不去）

**易错点**：

- ⚠️ 地图没加载出来 → **解决**：map_server 的 yaml_filename 由 launch 传 `map:=slam_map`，确认地图文件名正确且位于 maps 目录。
- ⚠️ footprint 比通道宽 → **解决**：机器人过不去是正常的（代价地图算过不可行），换宽通道或缩小 inflation。

---

## 步骤3：启动导航系统（60min）

**知识点讲解**：

**navigation.launch.py（3.5.1）**

- **原理**：一键启动：① bringup（驱动+里程计）→ ② lidar → ③ **localization**（map_server + amcl，加载地图并定位）→ ④ **navigation**（五大服务器）→ ⑤ RViz。地图路径直接从源码目录读取（`~/ros2_ws/src/xflz_eai_car1/slam/car1_slam/maps`），不依赖 install 目录。
- **源码对应**：`launch/navigation.launch.py` L13-69（launch_setup 中 maps_dir 硬编码、localization/navigation 两个 include、RViz 节点）。

**操作步骤**：

1. 启动导航：

```
source /opt/ros/humble/setup.bash && source ~/ros2_ws/install/setup.bash
ros2 launch car1_navigation navigation.launch.py map:=slam_map
```

2. 观察日志顺序：bringup 节点 → lidar → `map_server`（加载地图）→ `amcl` → 各 Nav2 服务器激活 → RViz 打开。
3. 检查节点与生命周期：

```
ros2 node list | grep -E "planner|controller|bt_navigator|behavior|amcl|map_server"
ros2 lifecycle get /planner_server    # 应为 active
ros2 topic echo /map --once           # 地图话题正常
```

4. RViz 确认：Map（黑色墙）显示、机器人模型出现在 map 原点附近（AMCL 初始 0,0,0）、LaserScan 显示。

**易错点**：

- ⚠️ 启动报"找不到 map 文件" → **解决**：确认 `map:=slam_map` 对应 `maps/slam_map.yaml` 存在；路径硬编码为源码目录，别改文件名。
- ⚠️ 机器人模型与地图墙体重叠 → **解决**：初始定位默认 0,0,0；若地图原点不在 (0,0)，用实验 4 的"2D Pose Estimate"手动修正。
- ⚠️ 某服务器一直 inactive → **解决**：查看该节点日志（`ros2 lifecycle get` + 对应 screen 输出），常见是地图未加载导致 planner 无法初始化。

---

## 步骤4：下发导航目标（90min）

**知识点讲解**：

**行为树流程（3.4.1 + 3.4.2）**

- **原理**：bt_navigator 加载行为树 XML（`navigate_to_pose_without_backup.xml`），流程大致：`收到目标 → 检查初始位姿 → 计算全局路径 → 跟踪路径 → 到达判定（xy_goal_tolerance 0.1m / yaw_goal_tolerance 0.1rad）→ 完成`。`navigators: ["navigate_to_pose", "navigate_through_poses"]` 对应单目标和多途经点两个导航器。
- **源码对应**：`nav2_params.yaml` L52-67（bt_navigator 配置与导航器注册）、L62-66（NavigateToPose / NavigateThroughPoses plugin）。

**下发目标两种方式（3.5.2）**

- **原理**：方式一 RViz "Nav2 Goal" 工具：点地图设定目标位置 + 拖拽设定朝向；方式二命令行 `ros2 action send_goal /navigate_to_pose`，适合脚本化与批量测试。
- **源码对应**：行为树 XML 中 `nav2_compute_path_to_pose_action_bt_node`（规划节点）与 `nav2_follow_path_action_bt_node`（跟踪节点）由 plugin_lib_names 注册（nav2_params.yaml L69-112）。

**操作步骤**：

1. RViz 方式：点击 "Nav2 Goal"，在空旷处点目标 + 拖朝向，观察：全局路径生成 → 小车转向 → 沿路径行驶 → 到达停稳。
2. 命令行方式（验证同样能到达）：

   ```
   ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
     "{pose: {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 1.0}, orientation: {w: 1.0}}}}"
   ```

   观察返回：`status: SUCCEEDED`（4）。
3. 连续测试 3 个不同目标点，记录：路径长度、是否绕路、到达误差（用 `ros2 topic echo /amcl_pose` 对比实际位置与目标点）。
4. 目标点选在墙内测试：观察路径规划失败（"No valid path"），理解代价地图的约束。

**易错点**：

- ⚠️ 点目标后无反应 → **解决**：检查 bt_navigator 是否 active、/map 是否加载、机器人是否已定位（RViz 看粒

子）。

- ⚠️ 小车在目标点附近反复打转 → **解决**：yaw 朝向未满足，调大 `yaw_goal_tolerance`（默认 0.1rad），或拖目标时把朝向对准。
- ⚠️ 到达判定失败（一直"接近中"）→ **解决**：xy_goal_tolerance 默认 0.10m，机械误差略大时可调大 0.15。

---

### 步骤5：避障与故障排查（90min）

**知识点讲解**：

**导航三要素（3.5.3）**

- **原理**：导航能不能成功，取决于三件事：① **地图质量**（实验 2 的墙不能重影）；② **定位准确**（AMCL 粒子收敛，见实验 4）；③ **速度约束合理**（velocity_smoother 的 max_velocity/max_accel 与小车物理匹配）。三缺一就翻车。
- **源码对应**：`nav2_params.yaml` L213-227 velocity_smoother（max_velocity [0.4,0,3.0]）。

**静态障碍避让**

- **原理**：路径上新增一个箱子，global_costmap 的 obstacle_layer 收到雷达点云后会把箱子标记为障碍，下次规划**自动绕行**；若箱子在行驶中出现，local_costmap 的 voxel_layer 实时标记，controller 现场绕。
- **源码对应**：global_costmap obstacle_layer L284-296（observation_sources: scan）、local_costmap voxel_layer L246-265。

**操作步骤**：

1. 静态障碍测试：在两点路径中点放一个箱子 → 下发目标 → 观察全局路径是否绕开箱子。
2. 动态障碍测试：让小车出发后，把箱子移到路径前方 → 观察小车现场减速绕行 → 移开箱子 → 观察路径恢复。
3. 故障排查演练（任选 2 个）：
    - 现象 A：小车贴墙走 → 检查 inflation_radius 是否过小、路径是否贴边
    - 现象 B：小车原地转圈 → 检查目标朝向与 yaw_goal_tolerance
    - 现象 C：规划失败 → 检查目标是否在墙内 / 地图是否最新
4. 记录每个现象的"现象 → 原因 → 解决"到实验报告。

**易错点**：

- ⚠️ 箱子挡住后小车反复尝试硬挤 → **解决**：等 5-10s 让 planner 重规划；仍不行检查 footprint 是否比缝隙大。
- ⚠️ 避障时小车抖动 → **解决**：controller_frequency 20Hz 正常，抖动多为速度上限过高（max_vel_x 0.3 → 0.2）或 acc 过大。
- ⚠️ 动态障碍移开后小车不恢复原路径 → **解决**：正常！只要到达目标就行，路径本来就是动态的。

## 3.7 注意事项

- ⚠️ 导航测试场地至少 3m×3m 且无贵重物品，防止失控碰撞。
- ⚠️ 全程保持人跟车，发现异常立即按底盘急停或 Ctrl+C。
- ⚠️ 每次改 nav2_params.yaml 需**重启导航 launch** 才生效。

- ⚠️ 导航与建图不要同时跑（都占用 /scan 与 TF，会互相干扰）。

## 3.8 实验总结

**完成成果**：

- 理解 Nav2 五大服务器与 Lifecycle 生命周期
- 掌握全局/局部代价地图分层与膨胀机制
- 独立完成"加载地图 → 设定目标 → 自主到达"闭环（RViz 与命令行两种方式）
- 完成静态/动态障碍避让测试与常见故障排查

**技能收获**：

- 技能1：Nav2 一键启动与节点/生命周期检查
- 技能2：RViz Nav2 Goal 与 ros2 action 双通道下发目标
- 技能3：代价地图与行为树故障定位

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| 地图没加载 | map:= 文件名错/文件缺失 | 检查 maps 目录与 launch 参数 |
| 目标无反应 | 未定位/节点未激活 | RViz 2D Pose Estimate 先定位；`ros2 lifecycle get` |
| 规划失败 | 目标在墙内/地图旧 | 目标点移到空旷处；重建图 |
| 到达判定失败 | 机械误差 > 容差 | 调大 xy_goal_tolerance/yaw_goal_tolerance |
| 撞墙 | inflation 太小/定位漂 | 调大 inflation_radius；重新定位 |

## 3.9 实验报告要求

**思考题**：

1. planner_server 与 controller_server 的分工是什么？为什么需要两层规划？
2. global_costmap 的 static/obstacle/inflation 三层各起什么作用？inflation_radius 调大对窄通道通行有何影响？
3. bt_navigator 用行为树而非硬编码 if-else 的好处是什么？NavigateToPose 与 NavigateThroughPoses 的区别？
4. 结合 velocity_smoother 参数，说明"急停急启"对底盘电机和定位的影响。

**评分标准**：Nav2 架构理解 20 分 + 参数解读 20 分 + 导航闭环操作 30 分 + 避障与故障排查 20 分 + 报告与思考题 10 分 = 100 分。

---

# 实验4：基于AMCL的自适应蒙特卡罗定位实验（8h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验源码位于 `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/config/nav2_params.yaml`（amcl 段），未复制到 `source-codes/EXP004/exp04/`。已通过 human_waiver 放行，内容基于 amcl 配置、`navigation.launch.py` 与 `slam_toobox_tutorial.md` 相关章节撰写。

**一句话读懂**：实验 3 里小车导航前"大概知道自己在哪"（默认 0,0,0）。但真实场景下小车可能被搬动、地图和现实对不上。这节课解决"**我到底在地图哪里**"——用 AMCL 粒子滤波，让小车拿雷达扫一扫，就自己猜出精确位置。

## 4.1 关联知识点

- 4.1.1 里程计漂移 > 概念 > 轮子打滑/累积误差，导航需要全局定位
- 4.1.2 雷达比对定位 > 原理 > 扫描与地图吻合处就是我的位置
- 4.2.1 粒子=假设 > 概念 > 每个粒子是一个"可能位置"的猜测
- 4.2.2 预测/更新/重采样 > 原理 > 三步循环收敛
- 4.2.3 自适应粒子数 > 概念 > 确定时减少、不确定时增加
- 4.3.1 AMCL参数 > 流程 > 粒子数/运动噪声/likelihood_field/update_min
- 4.3.2 初始位姿 > 概念 > set_initial_pose 或 RViz 2D Pose Estimate
- 4.4.1 观察粒子收敛 > 流程 > 绿箭头聚成一团即成功
- 4.4.2 kidnapped robot > 原理 > 被搬走后自动重新全局定位

## 4.2 实验目标

1. 理解里程计漂移问题与"雷达比对定位"的思想 — **原理理解**
2. 掌握粒子滤波"预测/更新/重采样"三步机制 — **原理理解**
3. 掌握 AMCL 参数配置与初始位姿设定方法 — **流程掌握**
4. 独立完成定位验证（粒子收敛观察）与 kidnapped robot 恢复测试 — **独立完成**
5. 能够通过参数调优解决"定位丢失/定位慢"问题 — **迁移应用**

## 4.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 具身智能实验箱B2 / 小车C1 + COIN-D6 雷达；已有地图（实验 2 产出） |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | Nav2 amcl（nav2_amcl）、map_server、nav2_msgs |
| 依赖 | `colcon build --packages-select car1_navigation`；地图位于 `slam/car1_slam/maps/` |
| 主入口 | `navigation/car1_navigation/config/nav2_params.yaml`（amcl 段）、`launch/navigation.launch.py`（localization 部分） |

## 4.4 实验难度

- 难度: ⭐ ⭐ ⭐ （综合性）
- 重点: 粒子滤波直觉理解；初始位姿设定；定位收敛验证
- 难点: 参数与收敛速度/精度的权衡；kidnapped robot 的恢复机制

## 4.5 实验现象

**快速验证流程**:

1. 启动导航（含定位）：`ros2 launch car1_navigation navigation.launch.py map:=slam_map`
2. RViz 中点击 **"2D Pose Estimate"**，在地图上点一下并拖出朝向（告诉 AMCL"我大概在这"）
3. 观察绿色箭头（粒子）从一大片逐渐**聚成一团**——定位成功
4. 遥控小车走几步再停下，观察粒子团是否跟随机身移动且保持收敛
5. 把小车**抱起来搬到另一个房间位置**再放下 → 粒子发散 → 几秒后自动重新聚拢（kidnapped robot 恢复）

**预期效果描述**:

- 初始粒子一大片散在地图上（说明"我不知道我在哪"）
- 给定初始位姿后粒子快速收敛成一团（位置误差 < 20cm）
- 小车移动时粒子团跟随，雷达扫描点（红色）与地图墙体重合
- 被搬走后粒子重新发散再收敛，`/amcl_pose` 更新为新位置

## 4.6 实验步骤

### 整体流程图




---

### 步骤1：理解定位问题（60min）

**知识点讲解**:

**里程计会漂移（4.1.1）**

- **原理**：里程计（odom）靠轮子编码器估算移动距离，但麦轮会打滑（尤其原地转圈打滑约 45%）、轮胎有磨损、地面有摩擦差异——这些误差会**累积**。跑 10 米可能偏几厘米，跑 100 米可能偏几十厘米。导航如果只用里程计，越走越不知道自己在哪。
- **白话类比**：蒙眼数步数走路，越走越不确定自己走到哪了——每步都有一点误差，攒多了就完全迷失。
- **源码对应**：本课程里程计由 bringup 的 EKF 融合轮速+IMU 发布 TF `odom→base_footprint`（见 `slam_toobox_tutorial.md` 3.4 节 EKF 输入分工）。

**雷达比对定位（4.1.2）**

- **原理**：解决漂移的办法是"**用环境说话**"：机器人举起雷达扫一圈，把扫描到的墙分布与地图比对——**扫描和地图最吻合的那个位置，就是机器人现在的位置**。这就是 AMCL 干的事。

- **白话类比**：你进了一家黑店，用手电筒照一圈，看到"左边有个吧台、右边有扇门"，对照记忆里的地图，立刻知道"哦我在吧台旁边"。

**操作步骤**：

1. 用 `ros2 topic echo /odometry/filtered` 观察里程计，理解它是"积分结果"。
2. 记录：为什么说里程计误差是"累积"的？雷达比对为什么能"清零"误差？
3. 画图：小车在房间里，雷达扫描如何与地图匹配出唯一位置。

**易错点**：

- ⚠️ 以为定位 = 里程计 → **解决**：里程计只能"短时间准"，定位（AMCL）负责"长期准"，两者配合。
- ⚠️ 以为需要 GPS → **解决**：室内无 GPS，AMCL 用激光雷达+地图就能定位。

---

## 步骤2：理解粒子滤波（90min）

**知识点讲解**：

**粒子 = 假设（4.2.1）**

- **原理**：AMCL 用一堆"粒子"表示"我可能在哪"的概率分布。每个粒子 = 一个 (x, y, yaw) 的猜测 + 一个权重（可信度）。一开始粒子乱撒（不知道在哪），越收敛粒子越集中。
- **白话类比**：就像考试蒙答案——一开始把 A/B/C/D 全涂上，涂对答案越多的位置越可信；最后只留最可能的那个选项。
- **源码对应**：`nav2_params.yaml` amcl 段 `max_particles: 2000`、`min_particles: 500`（粒子数量上下限）。

**三步循环（4.2.2）**

- **原理**：每一轮定位循环做三件事：
    1. **预测（Prediction）**：按里程计的运动量，把每个粒子"推着走"一步，模拟"如果我在这个位置，走这么多应该到哪"
    2. **更新（Update）**：拿一帧雷达扫描，与地图比对，给每个粒子**打分**——扫描吻合的打高分，不吻合的打低分
    3. **重采样（Resampling）**：淘汰低分粒子，复制高分粒子，粒子们"优胜劣汰"，逐步集中到真实位置附近
- **白话类比**：一群探路者：先按步数各自走（预测）→ 走到后用手电照墙对比地图，画对的地方举手（更新）→ 举对手的留下并复制，举错手的淘汰（重采样）。几轮下来，就剩下真位置附近的人。
- **源码对应**：`nav2_params.yaml` amcl 段 `update_min_a: 0.15`、`update_min_d: 0.15`——机器人转 0.15rad 或走 0.15m 才做一次更新循环（避免每帧都算，省算力）。

**自适应粒子数（4.2.3）**

- **原理**：粒子太多算力高、太少收敛慢。AMCL 用 KLD 采样自适应：**定位不确定时自动增加粒子**（加速收敛），**确定时自动减少粒子**（省算力）。这就是"自适应蒙特卡罗"的由来。
- **源码对应**：`nav2_params.yaml` amcl 段 `min_particles: 500` / `max_particles: 2000` 就是自适应的上下限。

**操作步骤**：

1. 在白纸上画图模拟三步循环：画 8 个粒子 → 走一步 → 打分 → 重采样，手工推演一轮。
2. 记录三个参数：max_particles / min_particles / update_min_a_d。
3. 思考（记录）：为什么重采样后粒子总数不变？淘汰低分后如何"补足"数量？

**易错点**：

- ⚠️ 粒子数越大越好？→ **解决**：2000 是上限，足够；更多只增算力不增精度。
- ⚠️ 混淆"预测"与"更新" → **解决**：预测靠里程计（不看环境），更新靠雷达（不看里程计），一个管"猜测位置"，一个管"验证位置"。

---

## 步骤3：配置 AMCL 参数（60min）

**知识点讲解**：

**核心参数（4.3.1）**

- **原理**：`nav2_params.yaml` 的 amcl 段关键参数：

| 参数 | 值 | 作用 |
| --- | --- | --- |
| min/max_particles | 500/2000 | 粒子数自适应范围 |
| alpha1~alpha5 | 0.2 | 运动模型噪声（越大越"怀疑里程计"） |
| laser_model_type | likelihood_field | 激光似然域模型（比 beam 模型更稳更快） |
| update_min_a / update_min_d | 0.15 | 转动/移动多少才更新粒子 |
| laser_max_range / min_range | 8.0 / 0.10 | 激光参与匹配的距离范围 |
| base_frame_id / global_frame_id / odom_frame_id | base_footprint / map / odom | 坐标系约定 |
| scan_topic | /scan | 激光话题 |
| set_initial_pose | true | 启动时用 initial_pose 初始化 |

- **源码对应**：`nav2_params.yaml` L5-50（amcl 段完整配置）。

**初始位姿（4.3.2）**

- **原理**：定位需要"从某处开始猜"。两种方式：① 配置 `set_initial_pose: true` + `initial_pose: [0,0,0]`（启动即从 map 原点开始猜）；② RViz 工具栏 **"2D Pose Estimate"** 手动点出初始位姿（更准确，推荐）。方式二本质是发布 `/initialpose` 话题。
- **源码对应**：`nav2_params.yaml` L45-50（set_initial_pose / initial_pose）。

**操作步骤**：

1. 打开 `nav2_params.yaml` amcl 段，对照上表逐项抄录并标注含义。
2. 将 `set_initial_pose` 改为 false 保存（体验"不预设位置"的纯全局定位，本步骤只改不重启，先记录预

期）。

3. 思考（记录）：如果完全不告诉 AMCL 初始位姿，粒子会撒在哪？收敛会更快还是更慢？

**易错点**：

- ⚠️ alpha1~5 全为 0 → **解决**：运动噪声为 0 表示"完全信里程计"，一旦打滑就定位失败；保持默认 0.2 左右。
- ⚠️ 改参数不生效 → **解决**：改 nav2_params.yaml 后必须**重启 navigation.launch**；运行中改用 `ros2 param set /amcl <param>` 临时生效（重启丢失）。

---

## 步骤4：定位验证（90min）

**知识点讲解**：

**观察粒子收敛（4.4.1）**

- **原理**：RViz 中 AMCL 粒子显示为绿色小箭头。**定位成功 = 箭头聚成一团**且与机器人模型重合；**定位失败 = 箭头散落在地图各处。**判定标准：机器人模型在地图墙线内、激光点云与墙体重合、粒子团半径 < 0.3m。
- **源码对应**：amcl 会发布 `particlecloud`（粒子）与 `amcl_pose`（最佳估计）两个话题。

**操作步骤**：

1. 启动导航：`ros2 launch car1_navigation navigation.launch.py map:=slam_map`
2. 观察默认行为：`set_initial_pose: true` 时粒子初始聚在 map 原点 (0,0,0)。
3. 用 "2D Pose Estimate" 在**机器人实际位置**重新点击设定（粒子重新聚拢）。
4. 遥控小车沿地图走几米并转向，随时观察：
    - 粒子团是否跟随机身移动（无粒子"掉队"）
    - 激光点云（红色）是否贴合地图墙线
    - `/amcl_pose` 输出位置是否平滑
5. 验证定位精度：让机器人停在一个可辨识位置（如墙角），对比 RViz 中模型位置与真实位置，记录误差。

**易错点**：

- ⚠️ 粒子分散不收敛 → **解决**：地图与实际环境不符（墙位置变了）或初始位姿点错方向（朝向反了）。重新点 "2D Pose Estimate"。
- ⚠️ 粒子收敛到错误位置 → **解决**：对称环境（如方形房间）会有"假收敛"；移动几步让扫描产生差异再观察，必要时重新定位。
- ⚠️ 粒子团抖动 → **解决**：`update_min_a/d` 过小导致频繁更新噪声大；或运动太快，放慢速度。

---

## 步骤5：kidnapped robot 与参数调优（90min）

**知识点讲解**：

**kidnapped robot（4.4.2）**

- **原理**：所谓"绑架问题"：机器人**被搬走**（或被人为推到别处）后，它还以为自己在老地方——所有粒子的位置假设全部失效。AMCL 的应对：当粒子权重持续偏低（扫描哪都不吻合）时，会自动**撒新粒子**（全局重定位），重新寻找与扫描匹配的位置，几秒内恢复。
- **白话类比**：你被蒙眼搬到另一个房间，睁眼发现周围环境全变了——于是重新环顾四周，根据吧台/窗户重新判

断"我在哪"。

- **源码对应**：`nav2_params.yaml` amcl 段 `recovery_alpha_fast: 0.0` / `recovery_alpha_slow: 0.0`（KLD 采样相关的快速/慢速恢复因子，控制重采样激进程度）。

**参数调优实验**

- **原理**：影响收敛速度与精度的可调项：
  - 收敛**慢** → 增大 `max_particles`（如 2000→3000）或调大 alpha（更怀疑里程计）
  - 定位**抖** → 减小 `laser_max_range`（忽略远距离噪声）或增大 `update_min_a/d`（少更新几次）
  - 精度**差** → 检查 `laser_model_type` 与 `z_hit/z_rand` 权重

**操作步骤**：

1. 绑架测试：将小车**抱起**移到房间另一处（保持朝向不变），观察：
   - 粒子先聚在"老位置"（假定位）
   - 雷达扫描与地图不符 → 粒子发散
   - 数秒后粒子重新聚拢到新位置，`/amcl_pose` 更新
   - 记录从"被搬走"到"恢复定位"的秒数
2. 测试"搬走+转向"：抱起并旋转 90° 放下，观察恢复是否更慢（航向更难猜）。
3. 调优对比（每次改参重启 launch）：
   - A 组：max_particles=2000（默认）→ 记录收敛时间
   - B 组：max_particles=3000 → 记录收敛时间与 CPU（`top -p $(pgrep -f amcl)`）
   - C 组：alpha1~5=0.5 → 观察定位稳定性变化
4. 结论写入报告：什么样的场景下需要调大粒子数？调大代价是什么？

**易错点**：

- ⚠️ 绑架后一直不恢复 → **解决**：环境特征太对称/地图太差，AMCL 难以区分位置；把小车移到特征明显处（墙角/门边）再测试。
- ⚠️ 恢复后粒子收敛到错误位置（对称房间） → **解决**：这是粒子滤波的已知局限；移动小车打破对称再观察。
- ⚠️ 调参后 CPU 飙升 → **解决**：max_particles 不要超 5000，RK3576 算力有限。

## 4.7 注意事项

- ⚠️ 绑架测试要**轻拿轻放**，确认雷达与 IMU 连接稳定，搬动前先停动机器人。
- ⚠️ 每次改 nav2_params.yaml 重启 launch；临时调参用 `ros2 param set` 但重启失效。
- ⚠️ 定位测试建议在特征明显的房间（有门、有柱子），不要在完全对称的大厅。
- ⚠️ 定位成功后再做导航（实验 3），否则导航会基于错误位置规划。

## 4.8 实验总结

**完成成果**：

- 理解里程计漂移与"雷达比对定位"思想
- 掌握粒子滤波预测/更新/重采样三步机制

- 配置 AMCL 参数并完成初始位姿设定
- 完成粒子收敛验证与 kidnapped robot 恢复测试
- 完成粒子数与噪声参数调优对比

**技能收获**：

- 技能1：RViz "2D Pose Estimate" 初始定位
- 技能2：粒子收敛状态的判读（聚拢/发散/假收敛）
- 技能3：AMCL 参数调优（粒子数/噪声/更新阈值）

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| 粒子不收敛 | 初始位姿错/地图不符 | 重新 2D Pose Estimate；检查地图 |
| 假收敛（错位置） | 对称环境 | 移动小车打破对称；重定位 |
| 收敛很慢 | 粒子少/运动模型置信高 | 调大 max_particles；调大 alpha |
| 定位抖动 | 更新过频/远距离噪声 | 调大 update_min_a/d；调小 laser_max_range |
| 绑架不恢复 | 地图差/特征对称 | 移到特征明显处重测 |

## 4.9 实验报告要求

**思考题**：

1. 为什么里程计误差是累积的？AMCL 的"雷达比对"为什么能消除累积误差？
2. 用你自己的话描述粒子滤波的"预测/更新/重采样"三步，并说明每步分别用了哪些传感器数据。
3. `update_min_a: 0.15` 的作用是什么？把它改小（如 0.01）对定位会有什么影响？
4. kidnapped robot 问题为什么难解？对称环境下的"假收敛"如何规避？
5. min_particles 与 max_particles 的"自适应"体现在哪里？调大 max_particles 的代价是什么？

**评分标准**：定位问题理解 15 分 + 粒子滤波原理 20 分 + AMCL 参数配置 20 分 + 定位验证实验 25 分 + kidnapped/调优实验 20 分 = 100 分。

---

# 实验5：局部路径规划与动态避障实验（8h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验源码位于 `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/config/nav2_params.yaml`（controller_server / local_costmap / behavior_server 段），未复制到 `source-codes/EXP004/exp05/`。已通过 human_waiver 放行，内容基于真实配置撰写。

**一句话读懂**：实验 3 里小车会"看地图走大路"，但路上突然滚来一个球、放下一只箱子怎么办？这节课专门练"**开车时的随机应变**"——用 DWB 局部规划器，让小车在行驶中实时躲开动态障碍，绕不过去还会自己转圈、后退重新想办法。

## 5.1 关联知识点

- 5.1.1 DWA速度窗口 > 概念 > 只考虑当前刹得住的速度组合
- 5.1.2 DWB评价函数 > 原理 > PathDist/GoalDist/BaseObstacle/RotateToGoal 加权
- 5.1.3 DWB参数 > 流程 > vx_samples/vtheta_samples/sim_time/max_vel
- 5.2.1 TEB弹性带 > 概念 > 把路径当弹性带用图优化调整
- 5.2.2 DWA vs TEB > 原理 > DWA 快而糙 / TEB 平滑可调时间
- 5.3.1 局部代价地图 > 流程 > 滚动窗口 + voxel + inflation
- 5.3.2 动态避障实验 > 流程 > 放障碍→绕行→移开→恢复
- 5.3.3 调优方向 > 原理 > 绕不过→调权重；抖→降速

## 5.2 实验目标

1. 理解 DWA/DWB 速度窗口思想与评价函数（critics）— **原理理解**
2. 理解 TEB 时间弹性带原理及其与 DWA 的差异 — **原理理解**
3. 掌握 local_costmap（滚动窗口/voxel/inflation）配置 — **流程掌握**
4. 独立完成动态避障实验并能够通过参数调优改善避障表现 — **独立完成**

## 5.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 具身智能实验箱B2 / 小车C1 + COIN-D6 雷达；地图（实验 2）+ 定位（实验 4）可用 |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | Nav2 dwb_controller、nav2_costmap_2d |
| 障碍物 | 纸箱/泡沫块（低矮安全，便于测试） |
| 主入口 | `navigation/car1_navigation/config/nav2_params.yaml`（controller_server / local_costmap 段） |

## 5.4 实验难度

- 难度: ⭐ ⭐ ⭐ （综合性）
- 重点: DWB 评价函数；动态避障实验；参数调优
- 难点: 调优方向判断（绕不过/抖动/贴障碍）

## 5.5 实验现象

**快速验证流程**:

1. 启动导航：`ros2 launch car1_navigation navigation.launch.py map:=slam_map`，并用 "2D Pose Estimate" 定位

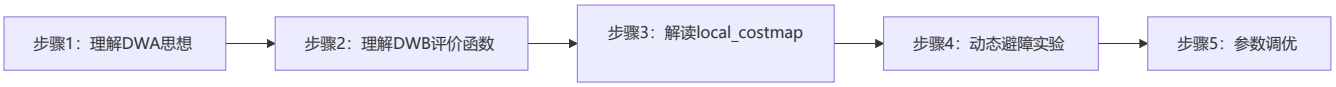
2. 下发一个**穿过房间中部**的目标点，让小车开始行驶
3. 小车行驶途中，把一个纸箱放到它正前方 1m 处
4. 观察：小车**减速** → **绕行**（离开全局路径走一段弧线）→ 绕过箱子 → 回到原路径 → 继续到达目标
5. 移开箱子再测一次：小车恢复走直线

**预期效果描述**：

- 避障时小车的速度曲线先降后升（voxel_layer 感知到障碍 → controller 减速）
- 绕行轨迹平滑（无急转弯），绕过后能重新贴合全局路径
- `ros2 topic echo /local_costmap/costmap_raw` 能看到障碍附近的代价从 0 升到 254（黑圈）
- 若障碍堵死通道，小车会停在原地转圈尝试（行为服务器介入）

# 5.6 实验步骤

## 整体流程图




---

## 步骤1：理解 DWA 速度窗口（60min）

**知识点讲解**：

**动态窗口法（5.1.1）**

- **原理**：DWA（Dynamic Window Approach）的核心思想：**不要考虑所有可能速度，只看"当前能刹得住"的速度。**对每对 (vx, ω) 组合：
    1. 先判断它是否满足加速度限制（acc_lim）
    2. 再判断用它刹车能否在障碍前停下
    3. 通过这两关的速度组成"**动态窗口**"，在窗口内采样生成模拟轨迹
    4. 用评价函数给每条轨迹打分，选最优执行
- **白话类比**：开车时你不会考虑"200km/h 冲过去"——先看能不能刹住（动态窗口），再从敢踩的速度里挑一条最好的路线。
- **源码对应**：`nav2_params.yaml` controller_server FollowPath（L133-171）：`acc_lim_x: 1.0`、`acc_lim_theta: 1.5`（加速度限制）、`vx_samples: 20`、`vtheta_samples: 40`（窗口内采样数）。

**DWB = DWA 的 Nav2 实现**

- **原理**：本项目局部规划器 plugin 是 `dwb_core::DWBLocalPlanner`，它把 DWA 思想模块化为多个可插拔的 **critics（评价器）**，每条候选轨迹由一组 critics 加权打分。
- **源码对应**：`nav2_params.yaml` L134（plugin）与 L161（critics 列表）。

**操作步骤**：

1. 抄录 FollowPath 的速度/加速度限制参数：max_vel_x/max_vel_theta/acc_lim_x/acc_lim_theta。
2. 理解"动态窗口"：用文字描述为什么 acc_lim 决定窗口大小。

3. 思考（记录）：max_vel_x=0.3（m/s）对室内小车意味着什么？（提示：3 秒走 1 米，安全优先）

**易错点**：

- ⚠️ 以为 DWA 会考虑所有速度 → **解决**：只考虑"刹得住"的速度，这是它"安全"的根本。
- ⚠️ 混淆全局路径与局部轨迹 → **解决**：全局路径是"目标线路"（长），局部轨迹是"接下来 1.5 秒怎么走"（短，sim_time: 1.5）。

---

## 步骤2：理解 DWB 评价函数（90min）

**知识点讲解**：

**critics 评分（5.1.2）**

- **原理**：每条候选轨迹由以下 critics 加权打分，得分越高越优先：

| critic | 作用 | 本项目权重(scale) |
|---|---|---|
| PathDist | 离全局路径的横向偏差 | 32.0 |
| PathAlign | 轨迹方向与全局路径对齐程度 | 32.0 |
| GoalDist | 离目标点的距离 | 24.0 |
| GoalAlign | 轨迹方向与目标朝向对齐 | 24.0 |
| BaseObstacle | 轨迹是否撞障碍（cost 采样） | 0.02 |
| RotateToGoal | 到目标时转正的效率 | 32.0 |
| Oscillation | 惩罚来回震荡（防抖动） | - |

- **白话类比**：教练给每个候选动作打分：走偏扣分（PathDist）、太远扣分（GoalDist）、会撞墙重扣（BaseObstacle）、来回磨蹭扣分（Oscillation）——总分最高的动作被选中。
- **源码对应**：`nav2_params.yaml` L161-171（critics 列表与 scale）。

**采样与模拟（5.1.3）**

- **原理**：`vx_samples: 20`（20 个线速度采样）× `vtheta_samples: 40`（40 个角速度采样）= 最多 800 条候选轨迹；每条用 `sim_time: 1.5`（模拟未来 1.5 秒）推演是否安全。
- **源码对应**：`nav2_params.yaml` L150-154（vx_samples/vtheta_samples/sim_time）。

**操作步骤**：

1. 抄录 critics 表并标注每个权重的含义。
2. 思考题（记录）：为什么 PathDist 权重 32 远大于 BaseObstacle 的 0.02？（提示：障碍惩罚靠"撞到就算失效"的硬约束，权重只需区分"远/近"）
3. 观察：启动导航后，`ros2 topic echo /local_plan`（若发布）查看规划出的局部轨迹。

**易错点**：

- ⚠️ 权重越大越好？→ **解决**：权重是**相对关系。**PathDist 大 = 更贴全局路径；BaseObstacle 的 0.02 与 cost 值相乘后仍能否决撞墙轨迹，因为撞墙轨迹的 BaseObstacle cost 是天文数字。

- ⚠️ Oscillation critic 让人困惑 → **解决**：它专门惩罚"左一下右一下"的摆动，防止小车在障碍前原地抖。

---

## 步骤3：解读 local_costmap（60min）

**知识点讲解**：

**局部代价地图构成（5.3.1）**

- **原理**：`local_costmap` 是跟着机器人走的 3m×3m 滚动窗口（`rolling_window: true`），只关心身边 3 米的障碍，更新频率 5Hz（比全局快，动态避障靠它）。两层：
  - **voxel_layer**：把雷达点云投成 3D 体素（`z_voxels: 16`、`max_obstacle_height: 2.0`），能在三维空间区分障碍高度
  - **inflation_layer**：把障碍向外膨胀 `inflation_radius: 0.18`，让轨迹自动与障碍保持距离
- **源码对应**：`nav2_params.yaml` L228-266（local_costmap：rolling_window、voxel_layer、inflation_layer）。

**代价地图可视化**

- **原理**：代价地图中：0=完全空闲、1-252=离障碍越近代价越高、254=障碍物、255=致命（不可入）。机器人轨迹会自动避开高代价区。
- **源码对应**：`publish_frequency: 2.0` 定期发布 costmap 话题供 RViz 显示。

**操作步骤**：

1. 抄录 local_costmap 关键参数：width/height 3、resolution 0.05、rolling_window true、inflation_radius 0.18。
2. RViz 添加 Display → By topic → /local_costmap/costmap_raw（Color 显示），观察行驶中代价地图如何跟随。
3. 在机器人旁 1m 放纸箱，观察代价地图中纸箱处出现"黑圈"（254）及周围渐变。
4. 思考（记录）：为什么局部地图窗口是 3×3 而不是整个房间？（提示：算力 + 只关心当前路况）

**易错点**：

- ⚠️ 代价地图不显示 → **解决**：确认 `always_send_full_costmap: true`（nav2_params.yaml 已设）且 RViz 话题正确。
- ⚠️ inflation 过大挡路 → **解决**：inflation_radius 0.18 已经考虑了小车半宽 0.18；调大后窄通道会判为不可行。

---

## 步骤4：动态避障实验（90min）

**知识点讲解**：

**动态避障闭环（5.3.2）**

- **原理**：完整避障链路：雷达 → voxel_layer 标记障碍 → costmap 更新（5Hz）→ controller 发现"当前轨迹撞了"→ 重采样生成绕行轨迹 → 绕过后按 PathDist 重新贴合全局路径。全程**无需重新规划全局路径**（除非绕行太远）。
- **白话类比**：高速上遇到前方事故——你不必重新查导航（全局路径），而是现场变道绕过，再回到原车道。

**操作步骤**：

1. 启动导航并定位，下发一个长距离目标（穿过房间）。
2. 小车行驶中，把纸箱放到其**正前方约 1m**处，保持不动。记录：
    - 避障开始时机（距障碍多远减速）
    - 绕行轨迹（离开全局路径的最大横向距离）
    - 绕过后是否回到原路径
3. 重复 3 次，记录避障成功率与每次的"最小安全距离"。
4. 挑战测试：
    - **窄通道**：把两个纸箱摆成 50cm 宽通道，看能否通过（footprint 宽 36cm）
    - **突然出现**：小车直线行驶时把箱子直接放到它面前 0.5m，观察急刹与绕行
5. 全程监控：`ros2 topic echo /cmd_vel` 观察速度变化曲线。

**易错点**：

- ⚠️ 避障时撞上箱子 → **解决**：箱子离机器人太近（< 局部窗口），或速度太快刹不住；放远一点 / 降 max_vel_x。
- ⚠️ 绕过障碍后"迷路"乱转 → **解决**：PathDist 权重不足，绕行后难以回贴全局路径；调大 PathDist.scale。
- ⚠️ 箱子 0.5m 突然出现急刹 → **解决**：机器人不是无限刹车性能；`acc_lim_x` 决定了减速能力，测试时保持合理距离。

---

## 步骤5：参数调优与 TEB 对比（90min）

**知识点讲解**：

**调优方向（5.3.3）**

- **原理**：遇到不同现象，改不同参数（每改必重启）：

| 现象 | 调优方向 |
| --- | --- |
| 绕不过障碍/撞障碍 | 调大 BaseObstacle.scale 或 inflation_radius；降低 max_vel_x |
| 抖动/左右摆 | 调大 Oscillation 惩罚；降低 sim_time；降速 |
| 绕行后回不来 | 调大 PathDist.scale / PathAlign.scale |
| 贴障碍太近 | 调大 inflation_radius；调大 BaseObstacle.scale |
| 到目标不转正 | 调大 RotateToGoal.scale / GoalAlign.scale |

- **源码对应**：`nav2_params.yaml` L161-171（critics scale 调整处）。

**TEB 时间弹性带（5.2.1 + 5.2.2）**

- **原理**：TEB（Time Elastic Band）把"路径"看成一根**弹性带**，用图优化同时调整**轨迹点位置**和**到达时间**，目标函数包括：贴路径、避障碍、时间最短、速度平滑。与 DWA 差异：DWA 只模拟"眼前 1.5 秒"的轨迹（快、低算力）；TEB 优化整条局部路径（平滑、可加时间约束）。本项目默认用 DWB，TEB 是可选替代（了解原理即可）。

- **源码对应**：本项目 nav2_params.yaml 未启用 TEB（`controller_plugins: ["FollowPath"]` → DWB）；TEB 为知识扩展。

**操作步骤**：

1. 调优实验 A：把 `BaseObstacle.scale` 从 0.02 调到 0.1，重启，重复"正前方放箱"测试，记录最小安全距离变化。
2. 调优实验 B：把 `max_vel_x` 从 0.3 调到 0.2，重复测试，对比避障成功率与到达耗时。
3. 把最优参数写回 nav2_params.yaml，记录"参数 → 现象"对照表（至少 3 组）。
4. 画一张 DWA vs TEB 对比表（原理/速度/平滑度/适用场景），并说明本项目为何选 DWB。
5. 写小结：给你的小车调出一组"又快又稳"的参数，附实验数据。

**易错点**：

- ⚠️ 一次只改一个参数 → **解决**：同时改多个无法定位是哪个起了作用；每次单变量对比。
- ⚠️ 参数越"激进"越好 → **解决**：权重拉满会导致路径僵化/无法变通；调参目标是"够用且稳健"。
- ⚠️ 忘记重启 → **解决**：controller_server 参数在启动时读取，改后必须重启 navigation.launch。

## 5.7 注意事项

- ⚠️ 障碍物用**低矮纸箱/泡沫**，不要用易碎或沉重物体。
- ⚠️ 动态避障测试时**手远离小车路径**，人站在开阔侧随时准备按急停。
- ⚠️ 每次调参前记录"当前参数 → 当前现象"，便于对比与回退。
- ⚠️ 窄通道测试前确认 footprint（0.18 半宽）能通过，否则属于正常不可行。

## 5.8 实验总结

**完成成果**：

- 理解 DWA 动态窗口思想与 DWB 七大 critics 评价函数
- 理解 TEB 时间弹性带原理及与 DWA 的差异
- 掌握 local_costmap（滚动窗口/voxel/inflation）配置与可视化
- 完成动态避障实验（正前方/窄通道/突然出现）与参数调优

**技能收获**：

- 技能1：DWB critics 权重调优方法
- 技能2：代价地图可视化解读（0/254 代价分布）
- 技能3：避障实验设计（单变量对比）

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| 避障撞箱 | 速度太快/距障太近 | 降 max_vel_x；放远障碍 |
| 绕行后回不来 | PathDist 权重低 | 调大 PathDist.scale |
| 行驶抖动 | Oscillation 惩罚不足 | 调大振荡惩罚；降速 |

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| 贴障碍走 | inflation 太小 | 调大 inflation_radius |
| 窄通道过不去 | footprint 宽于通道 | 属正常；换宽通道 |

## 5.9 实验报告要求

**思考题**:

1. 用一句话解释"DWA 为什么只考虑刹得住的速度"？acc_lim_x 在这个判断中起什么作用?
2. 列举 DWB 的 4 个 critics 并说明各自作用；为什么 BaseObstacle 权重那么小却仍能阻止撞墙轨迹?
3. local_costmap 为什么用 3×3 滚动窗口而不是全图？voxel_layer 相比二维层有什么优势?
4. 对比 DWA/DWB 与 TEB：各自适用什么场景？本项目为什么默认选择 DWB?
5. 你调优实验中"现象→参数"的对应关系是什么？请列出至少 2 组并解释原因。

**评分标准**：DWA 原理 15 分 + DWB 评价函数 20 分 + local_costmap 配置 15 分 + 动态避障实验 30 分 + 参数调优与对比 20 分 = 100 分。

---

# 实验6：基于激光雷达的单点与多点导航避障实验（8h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验源码位于 `ros2_ws/src/xflz_eai_car1/navigation/car1_navigation/`，未复制到 `source-codes/EXP004/exp06/`。已通过 human_waiver 放行，内容基于 `config/nav2_params.yaml`（bt_navigator / behavior_server / progress_checker 段）与 `launch/navigation.launch.py` 撰写。

**一句话读懂**：前面我们"点一个目标，到一处"。真实任务往往是"**先去 A 放货，再去 B 取货，最后回 C**"。这节课用两个 Nav2 导航器（NavigateToPose / NavigateThroughPoses），让小车**连续完成多个目标**，而且中途有障碍也能一路安全走完。

## 6.1 关联知识点

- 6.1.1 导航Action > 概念 > /navigate_to_pose 单目标、/navigate_through_poses 多点
- 6.1.2 单点导航 > 流程 > ros2 action send_goal 命令
- 6.2.1 多点导航 > 流程 > 一次下发多个途经点顺序经过
- 6.2.2 任务调度 > 原理 > 目标序列 + 到达判定 + 失败重试
- 6.3.1 评估指标 > 流程 > 到达率/平滑度/避障响应/停车次数
- 6.3.2 安全兜底 > 原理 > 进度检查 + 行为服务器恢复

## 6.2 实验目标

1. 理解 ROS2 Action 通信机制与两个导航器的区别 — **原理理解**
2. 掌握单点导航与多点连续导航的命令行操作 — **流程掌握**
3. 掌握导航任务调度与失败重试的基本方法 — **流程掌握**
4. 能够评估多点导航的鲁棒性并处理异常（卡住/超时）— **迁移应用**

## 6.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 具身智能实验箱B2 / 小车C1 + COIN-D6 雷达；地图 + 定位可用 |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | Nav2 bt_navigator、nav2_msgs（NavigateToPose/NavigateThroughPoses action） |
| 依赖 | 完成实验 3/4；导航 launch 正常运行 |
| 主入口 | `navigation/car1_navigation/launch/navigation.launch.py`、`config/nav2_params.yaml` |

## 6.4 实验难度

- 难度: ⭐ ⭐ ⭐ （综合性）
- 重点: Action 下发与多点连续导航
- 难点: 连续导航的调度与异常恢复

## 6.5 实验现象

**快速验证流程**:

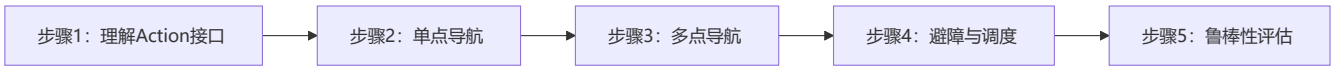
1. 启动导航：`ros2 launch car1_navigation navigation.launch.py map:=slam_map`，完成定位
2. 单点导航：`ros2 action send_goal /navigate_to_pose ...`（或 RViz Nav2 Goal），观察小车到达
3. 多点导航：先启动客户端脚本，再同时发布 A/B/C 三个途经点，观察小车**依次经过**三个点最终停下
4. 途中放纸箱：多点导航时小车在点与点之间自动避障
5. 堵死通道测试：观察小车卡住 → 行为服务器 spin/backup 恢复 → 重试

**预期效果描述**:

- 单点：小车从起点直线/绕行到达目标，`ros2 action send_goal` 返回 SUCCEEDED
- 多点：小车轨迹依次经过 A→B→C（可用 RViz 的 Path 或记录 /amcl_pose 验证）
- 避障：途经点之间遇到障碍自动绕行，不中断任务
- 异常：通道被堵死时小车尝试恢复动作（原地转/后退），恢复失败则任务失败（ABORTED），不会盲目硬闯

## 6.6 实验步骤

### 整体流程图




---

### 步骤1：理解 ROS2 Action 与导航器（60min）

**知识点讲解**:

**Action 通信机制（6.1.1）**

- **原理**：ROS2 的 **Action** 是"话题+服务"的组合升级：请求方发**目标（goal）** → 执行方持续回报**反馈（feedback）** → 完成后返回**结果（result）**。对导航这种"要跑很久、要随时看进度"的任务非常合适。
- **白话类比**：Topic 像"对讲机广播"（单向）、Service 像"打电话"（一问一答）、Action 像"派快递"（下单→物流跟踪→签收确认）。
- **源码对应**：`nav2_params.yaml` bt_navigator `navigators: ["navigate_to_pose", "navigate_through_poses"]`（L62）注册了两个导航器；`nav2_msgs/action/NavigateToPose`、`NavigateThroughPoses` 是动作定义。

**两个导航器（6.1.2 + 6.2.1）**

- **原理**：
  - `navigate_to_pose`：只有一个最终目标点（位置+朝向），行为树流程：计算路径 → 跟踪 → 到达
  - `navigate_through_poses`：有一串**途经点**（intermediate poses），机器人**必须依次经过每个点**，最后停在最后一个点；常用于"巡检路线"等任务
- **源码对应**：`nav2_params.yaml` L63-66（NavigateToPoseNavigator / NavigateThroughPosesNavigator plugin）。

**操作步骤**：

1. 查看 action 定义：

```
ros2 interface show nav2_msgs/action/NavigateToPose
# 看 goal（pose）/ feedback（distance_remaining）/ result（result）
```

2. 对比两个 action 的 goal 结构（NavigateThroughPoses 是 `poses[]` 数组）。
3. 画图：Action 三部分（goal/feedback/result）在导航中的语义。

**易错点**：

- ⚠️ 混淆 service 与 action → **解决**：导航是"长时间+可取消+有反馈"的任务，用 action 而非 service。
- ⚠️ 以为 send_goal 是阻塞的 → **解决**：`ros2 action send_goal` 默认等待完成（--feedback 可看进度）；`&` 后台执行需自测。

---

## 步骤2：单点导航（60min）

**知识点讲解**：

**单点导航命令（6.1.2）**

- **原理**：命令行下发一个 PoseStamped 目标，bt_navigator 自动完成规划+执行。frame_id 必须为 **map**（全局坐标系），位置单位米，朝向用四元数（w=1 表示朝向 map 的 +X 方向）。
- **源码对应**：目标经过行为树 `nav2_compute_path_to_pose_action_bt_node` → `nav2_follow_path_action_bt_node`（plugin_lib_names 已注册，nav2_params.yaml L69-112）。

**操作步骤**：

1. 启动导航并定位（实验 3/4 的完整环境）。
2. 单点导航（目标 map 系 (1.0, 0.5)）：

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 0.5}, orientation:
{w: 1.0}}}}"
```

3. 加 --feedback 观察实时进度：

```
ros2 action send_goal --feedback /navigate_to_pose ...
# feedback.distance_remaining 逐渐减小
```

4. 连续下发 5 个不同目标点，记录每个：到达用时、最终位置误差（`ros2 topic echo /amcl_pose` 对比）。
5. 目标点放在墙内测试：观察 action 返回 ABORTED（规划失败），理解"目标不可达"。

**易错点**：

- ⚠️ frame_id 写错（如写 odom）→ **解决**：目标必须是 `map` 系，否则机器人会去错误位置。
- ⚠️ 目标在墙内仍返回 SUCCEEDED？→ **解决**：不会，规划失败会 ABORTED；若在墙边，xy_goal_tolerance 内可能"够到"。
- ⚠️ 朝向四元数写错 → **解决**：想朝 +X 用 w=1；朝 -Y 用 (0,0,-0.707,0.707)；可用 `ros2 run tf2_ros tf2_echo` 验证。

---

## 步骤3：多点连续导航（90min）

**知识点讲解**：

**navigate_through_poses（6.2.1）**

- **原理**：一次下发多个途经点 `poses[]`，机器人必须**依次经过**每个点（点与点之间也避障），最后停在最后一个点。适合"巡检、送货路线"。
- **源码对应**：`nav2_params.yaml` L66（NavigateThroughPosesNavigator）；行为树 `nav2_compute_path_through_poses_action_bt_node`（L71）。

**操作步骤**：

1. 下发 3 个途经点（A(0,0.5) → B(1,1) → C(1.5,0.5)）：

```
ros2 action send_goal /navigate_through_poses nav2_msgs/action/NavigateThroughPoses \
  "{poses: [{header: {frame_id: 'map'}, pose: {position: {x: 0.0, y: 0.5},
orientation: {w: 1.0}}},
            {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 1.0},
orientation: {w: 1.0}}},
            {header: {frame_id: 'map'}, pose: {position: {x: 1.5, y: 0.5},
orientation: {w: 1.0}}}]"
```

2. 观察小车轨迹是否**依次经过三点**（记录 /amcl_pose 时间序列，绘制轨迹）。
3. 对比测试：改用三个**单点** navigate_to_pose 连续下发，对比：
   - 总耗时 vs 多点导航
   - 轨迹差异（多点会强制经过中间点，单点可能"抄近路"）
4. 验证"必须经过"语义：把 B 点放到障碍区，观察多点导航卡住或绕行，而单点导航不受影响。

**易错点**：

- ⚠️ 以为多点 = 只去终点 → **解决**：中间点是**必经点**，机器人会刻意经过（这正是它与单点的本质区别）。
- ⚠️ 途经点间距太小（<0.3m）→ **解决**：相邻点太近会导致行为树到达判定混乱，保持间距 >0.5m。

---

## 步骤4：任务调度与避障（90min）

**知识点讲解**：

**任务调度（6.2.2）**

- **原理**：连续任务 = "目标序列管理 + 逐个执行 + 异常处理"。每段导航看 **action 返回结果**：SUCCEEDED→下一个目标；ABORTED→记录失败并重试或跳过。到达判定由 `general_goal_checker` 完成（xy_goal_tolerance 0.10m / yaw_goal_tolerance 0.1rad）。
- **源码对应**：`nav2_params.yaml` L128-132（general_goal_checker）。

**安全兜底（6.3.2）**

- **原理**：导航过程中如果机器人"长时间没挪窝"，`progress_checker` 判定卡住（`required_movement_radius: 0.1` / `movement_time_allowance: 10.0`，10 秒没动 10cm 算卡住）→ behavior_server 介入执行恢复动作：**spin**（原地转一圈重新找路）、**backup**（后退）、**drive_on_heading**（沿朝向直行）→ 恢复后重试路径。恢复无效则任务 ABORTED。
- **白话类比**：导航像送外卖——骑手迷路了先原地转圈看看（spin），不行倒车出来（backup），还不行就报告"这单送不了"（ABORTED），而不是硬闯。
- **源码对应**：`nav2_params.yaml` L121-127（progress_checker）、L185-212（behavior_server：spin/backup/drive_on_heading/assisted_teleop/wait）。

**操作步骤**：

1. 写一个简单的 Python 调度脚本（伪代码，改造成可运行的）：

```
# 依次导航到目标列表，失败自动重试 1 次
goals = [g1, g2, g3]
for g in goals:
    result = send_navigate_to_pose(g)    # 阻塞等待完成
    if result == SUCCEEDED:
        print(f"目标 {g} 完成")
    else:
        print(f"目标 {g} 失败，重试")
        result = send_navigate_to_pose(g)
```

2. 通道堵死测试：把 3 个纸箱堵住必经通道 → 下发目标 → 观察：
    - progress_checker 卡住判定（日志 `Failed to make progress`）
    - behavior_server 执行 spin/backup
    - 尝试恢复失败 → ABORTED
3. 记录完整时间线：卡住时刻 → 恢复动作 → 结果。
4. 对比验证：只堵一半（留 60cm 通道），观察能否绕过继续（避障生效，无需恢复动作）。

**易错点**：

- ⚠️ 误以为卡住后无限重试 → **解决**：恢复动作有限次，无效则 ABORTED，这是安全设计。
- ⚠️ 调度脚本忘处理 ABORTED → **解决**：真实任务必须检查 action 结果，失败要记录与重试，否则任务"静默失败"。

---

## 步骤5：鲁棒性评估（90min）

**知识点讲解**：

**评估指标体系（6.3.1）**

- **原理**：导航系统"好不好"用四个指标量化：

| 指标 | 定义 | 本项目合格线（参考） |
| --- | --- | --- |
| 到达率 | 成功到达目标的任务占比 | ≥ 80% |
| 路径平滑度 | 路径转弯次数/轨迹抖动 | 无急转、无明显抖动 |
| 避障响应 | 从障碍出现到开始绕行的时间 | < 2s |
| 中途停车 | 非终点停车次数（卡住触发） | ≤ 1 次/任务 |

- **源码对应**：progress_checker（卡住检测）与 goal_checker（到达判定）共同决定这些指标。

**操作步骤**：

1. 设计一条 3 点巡检路线（A→B→C），连续跑 5 遍，记录：
    - 到达率（成功/总数）
    - 每段平均耗时
    - 中途停车次数
    - 是否有 recovery 动作触发
2. 加入 1 个移动障碍（人走动），再跑 5 遍，对比到达率与避障响应。
3. 数据汇总表：

| 组别 | 到达率 | 平均耗时 | 停车次数 | 避障响应 |
| --- | --- | --- | --- | --- |
| 无障碍 5 次 | | | | - |
| 有动障 5 次 | | | | |

4. 写鲁棒性小结：这套小车+Nav2 配置在什么条件下可靠，什么条件下会失效（如对称大厅、超窄通道）。

**易错点**：

- ⚠️ 只测一次就下结论 → **解决**：至少 5 次取统计，导航是有随机性的。
- ⚠️ 动障测试时人挡住传感器 → **解决**：人不要在雷达正前方静止超过 0.5s，保持走动，否则会被当成静态障碍。

## 6.7 注意事项

- ⚠️ 多点导航测试场地至少 3m×4m，多点间留出转弯空间。
- ⚠️ 调度脚本注意加"目标可达"预检查（用 costmap 判断），避免明知不可达还发。
- ⚠️ 恢复动作测试时人远离小车，spin 旋转半径内不要站人。
- ⚠️ 每次评估跑多遍取统计，单次现象不做结论。

## 6.8 实验总结

**完成成果**：

- 理解 ROS2 Action 机制与两个导航器的语义差异
- 掌握单点（navigate_to_pose）与多点（navigate_through_poses）导航命令
- 完成任务调度（目标序列 + 失败重试）与避障测试
- 完成卡住恢复（progress_checker + behavior_server）观察与鲁棒性评估

**技能收获**：

- 技能1：ros2 action 下发与反馈查看
- 技能2：多点导航调度脚本编写
- 技能3：导航鲁棒性量化评估（到达率/停车/避障响应）

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| 多点导航不按顺序 | 途经点配置错误 | 检查 poses[] 顺序与 frame_id |
| 中间点被跳过 | 到达判定容差过大 | 调小 xy_goal_tolerance |
| 一直卡住不恢复 | progress_checker 未触发 | 检查 movement_time_allowance |
| 恢复后仍失败 | 通道真堵死 | 属正常 ABORTED，勿盲目重试 |
| 调度脚本无响应 | 未检查 action 结果 | 加 result 检查与日志 |

## 6.9 实验报告要求

**思考题**：

1. Action 相比 Topic/Service 有什么优势？导航为什么必须用 Action？
2. navigate_to_pose 与 navigate_through_poses 的语义差异是什么？各自适合什么任务场景？
3. progress_checker（10s/10cm）如何判断"卡住"？为什么需要这个机制？
4. 多点导航中某个点失败，你的调度策略是什么？（重试/跳过/终止）请说明权衡。
5. 结合鲁棒性数据，分析这套系统在什么场景下会失效？如何改进？

**评分标准**：Action 与导航器理解 15 分 + 单点导航 20 分 + 多点导航 25 分 + 调度与避障 25 分 + 鲁棒性评估 15 分 = 100 分。

---

# 实验7：基于RTAB-Map的三维视觉SLAM导航实验（8h）

> ⚠️ **[待源码核实 / WAIT-SOURCE]** 本实验为**原理+通用流程**型实验：spec 标注 `source_code_path: source-codes/exp07/`、notes=无资源（仓库暂缺 RTAB-Map 专用源码）。已通过 human_waiver 放行。内容基于课程硬件 HP60C 深度相机（`examples/camera/HP60C/`）、Nav2 导航栈与通用 RTAB-Map 知识撰写；后续补充源码后需触发 SourceCodeAnalysisAgent 并移除豁免。

**一句话读懂**：前 6 个实验，小车眼里世界是"一张平面图"（俯视图，像户型图）。这节课让小车"长出一双立体眼睛"——用 RGB-D 深度相机 + RTAB-Map，构建**三维点云/八叉树地图**，让机器人理解高度、层次、悬空障碍，为"分楼层导航、抓取定位"打基础。

## 7.1 关联知识点

- 7.1.1 RTAB-Map框架 > 概念 > Real-Time Appearance-Based Mapping 实时外观建图
- 7.1.2 视觉特征回环 > 原理 > 用图像特征/词袋做回环检测
- 7.2.1 RGB-D相机 > 概念 > 彩色图+深度图，(u,v) 能算 3D 点
- 7.2.2 点云/OctoMap > 概念 > 海量 3D 点 / 体素化可压缩地图
- 7.3.1 三维建图流程 > 流程 > 启动相机→RTAB-Map→操控→保存
- 7.3.2 三维导航 > 流程 > OctoMap 作为代价地图输入
- 7.4.1 2D vs 3D > 原理 > 二维省资源 / 三维信息全算力高

## 7.2 实验目标

1. 理解视觉 SLAM 与 RTAB-Map 的基本原理（特征、词袋回环）— **原理理解**
2. 理解 RGB-D 相机的工作原理与点云/八叉树地图的表示 — **原理理解**
3. 掌握三维建图与三维导航的通用流程 — **流程掌握**
4. 能够对比二维与三维 SLAM 的适用场景并做选型分析 — **迁移应用**

## 7.3 实验环境

| 类别 | 配置 |
| --- | --- |
| 硬件 | 具身智能实验箱B2 / 小车C1 + HP60C RGB-D 深度相机（`examples/camera/HP60C/`） |
| OS | Ubuntu 22.04 + ROS 2 Humble |
| 核心库 | rtabmap_ros（ROS2 版）、depth_image_proc、octomap_server、rviz2 |
| 依赖 | HP60C 相机驱动（ascamera）能发布 RGB + Depth 话题；Nav2 可用（实验 3） |
| 主入口 | 通用流程：`ros2 run rtabmap_slam rtabmap` + 相机驱动；本项目为原理演示型 |

> 说明：由于仓库暂无 RTAB-Map 专用 launch/配置源码，本实验以原理讲解 + 通用命令 + 演示观察为主；若后续补充源码，将按源码更新操作步骤。

## 7.4 实验难度

- 难度: ⭐ ⭐ ⭐ （综合性，进阶）

- 重点: 三维地图表示；RTAB-Map 原理；三维流程
- 难点: 三维 SLAM 的算力与工程复杂度理解（本实验以理解为主）

## 7.5 实验现象

**快速验证流程**（有 HP60C 相机时，无相机可只做原理+观看演示）：

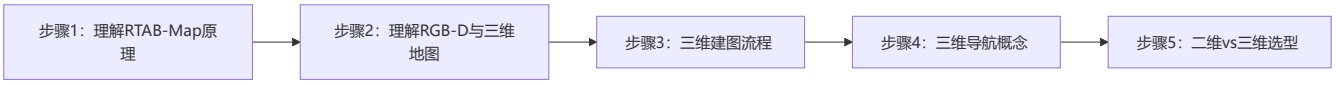
1. 启动相机：`ros2 launch ascamera hp60c.launch.py`（或按相机文档启动），确认 `/camera/rgb/image_raw` 与 `/camera/depth/image_raw` 有数据
2. 启动 RTAB-Map：`ros2 run rtabmap_slam rtabmap --ros-args -p subscribe_rgbd:=true ...`
3. 遥控小车在室内慢慢走一圈，RViz 中观察三维点云地图"长"出来
4. 保存三维地图/八叉树，并在 RViz 中旋转观察立体结构（墙、桌椅、悬空物）

**预期效果描述**：

- RViz 显示彩色三维点云：墙面有颜色、桌椅轮廓清晰、能看出高度差
- 回环时点云地图发生"咔"地一次大对齐（累积漂移被消除）
- 八叉树地图体素清晰可旋转，悬空物（如桌面、灯）与地面分层明显
- 对比实验 2 的二维地图：三维地图信息量大得多，但 RViz 画面也更"重"

## 7.6 实验步骤

### 整体流程图




---

### 步骤1：理解 RTAB-Map 原理（90min）

**知识点讲解**：

**RTAB-Map 框架（7.1.1）**

- **原理**：RTAB-Map（Real-Time Appearance-Based Mapping）是**基于外观（图像内容）的实时建图**框架，支持 RGB-D 与双目相机。核心组件：
  - **前端里程计**：用相邻帧图像特征匹配（如 ORB）估计相机运动
  - **局部地图**：短期内的关键帧与观测
  - **回环检测**：用**词袋模型（Bag of Words）**在内存中检索"和当前帧最像的历史帧"，命中即回环
  - **图优化**：g2o/GTSAM 优化位姿图，消除漂移
- **白话类比**：二维 SLAM 用"墙的轮廓"认路，RTAB-Map 用"照片里的特征点"认路——看到"熟悉的那面墙的照片"就知道"我来过这"。
- **与 slam_toolbox 的关系**：两者都是图优化 SLAM，只是传感器不同（激光 vs 相机），回环手段不同（扫描匹配 vs 图像词袋）。

**视觉特征与词袋（7.1.2）**

- **原理**：每张图像提取几百个**特征点**（ORB：关键点+描述子）。把特征"聚类"成词汇，一帧图像 = 一组词汇的集

合（词袋向量）。回环检测时比较当前词袋与历史词袋的相似度，超过阈值即触发回环。

- **白话类比**：就像用"照片里出现了吧台+红椅+吊灯"判断"我回到了这家咖啡馆"，而不是逐像素比对整张照片。

**操作步骤**：

1. 画出 RTAB-Map 的模块图：图像 → 特征提取 → 里程计 → 局部地图 → 词袋回环 → 图优化 → 地图。
2. 对比表（记录）：slam_toolbox（实验 2）与 RTAB-Map 在传感器/回环手段上的差异。

| 维度 | slam_toolbox | RTAB-Map |
| --- | --- | --- |
| 传感器 | 激光雷达 | RGB-D 相机 |
| 回环手段 | 扫描匹配 | 图像词袋 |
| 地图 | 二维栅格 | 三维点云/八叉树 |

**易错点**：

- ⚠️ 以为 RTAB-Map 不用激光 → **解决**：它也能融合激光，但核心输入是图像；纯激光场景还是用 slam_toolbox 更高效。
- ⚠️ 以为词袋 = 深度学习 → **解决**：词袋是经典特征聚类（ORB+KMeans），不是神经网络。

---

## 步骤2：理解 RGB-D 相机与三维地图（90min）

**知识点讲解**：

**RGB-D 相机（7.2.1）**

- **原理**：HP60C 这类 RGB-D 相机同时输出：**彩色图**（每个像素的颜色 RGB）与**深度图**（每个像素到相机的距离 d，单位毫米）。有了相机内参 (fx, fy, cx, cy)，每个像素 (u,v) 都能反投影成三维点：

```
X = (u - cx) * d / fx
Y = (v - cy) * d / fy
Z = d
```

- **白话类比**：普通相机只记"那里有个东西"，深度相机还记"那个东西离我多远"——有了方向和距离，就能拼出立体世界。
- **源码对应**：HP60C 驱动位于 `examples/camera/HP60C/src/ascamera/`（ascamera 节点发布 RGB/Depth）；深度图话题如 `/camera/depth/image_raw`。

**三维地图表示（7.2.2）**

- **原理**：两种常见表示：
  - **点云地图**：海量 3D 点（每帧图像几十万个），直观但冗余、无法编辑
  - **八叉树地图（OctoMap）**：把空间递归切成立方体（体素），叶子节点记录占据概率，**可压缩、可增量更新、支持查询**——导航/避障常用
- **白话类比**：点云是"一堆沙子"（每个颗粒一个点），OctoMap 是"乐高积木"（规则的小方块，能数得清哪块有人哪块空）。

**操作步骤**：

1. 确认相机驱动与话题（有相机时）：

```
ros2 topic hz /camera/depth/image_raw    # 深度图频率
ros2 topic hz /camera/rgb/image_raw      # 彩色图频率
```

2. 用 `ros2 run rqt_image_view rqt_image_view /camera/depth/image_raw` 查看深度图（近红远蓝）。
3. 用 numpy 手算一次反投影（给定内参 + 像素点），把 (u,v,d) 转成 (X,Y,Z)。
4. 记录：点云与 OctoMap 各适合什么用途。

**易错点**：

- ⚠️ 深度图是全黑的 → **解决**：深度相机对反光/远距离失效（返回 0）；保持 0.5-3m 有效距离。
- ⚠️ 内参不知道 → **解决**：相机话题 /camera_info 里有 fx/fy/cx/cy，`ros2 topic echo /camera/depth/camera_info`。
- ⚠️ 反投影公式记反 → **解决**：验证：把像素放图像中心 (cx,cy)，应得到 X≈0、Y≈0、Z=d。

---

## 步骤3：三维建图流程（120min）

**知识点讲解**：

**通用三维建图流程（7.3.1）**

- **原理**：三维建图 = 相机驱动 + RTAB-Map 节点 + 操控 + 可视化：

```
1. 启动相机（RGB + Depth + 内参）
2. 启动 RTAB-Map（subscribe_rgbd:=true，吃 RGB-D 话题）
3. 遥控小车缓慢走遍场景（转角慢、多停留、回环闭合）
4. RViz 观察三维点云逐渐成形
5. 保存地图：rtabmap 数据库（.db）+ 导出 OctoMap（octomap_server）
```

- **白话类比**：就像用手机 3D 扫描 App 扫房间——慢慢绕、别太快、绕回起点，扫出来才完整。
- **源码对应**：本项目仓库暂无 RTAB-Map 专用 launch，通用命令：

```
# 相机（以 HP60C 为例，launch 名以实际为准）
ros2 launch ascamera hp60c.launch.py
# RTAB-Map（通用）
ros2 run rtabmap_slam rtabmap --ros-args -p subscribe_rgbd:=true \
  -p rgb_topic:=/camera/rgb/image_raw -p depth_topic:=/camera/depth/image_raw
```

**操作步骤**（有相机条件）**：

1. 启动相机，确认三话题（rgb/depth/camera_info）正常。
2. 启动 RTAB-Map 与 RViz（Add → PointCloud2 显示三维点云）。
3. 遥控小车**低速**走一圈（闭环），转角处停留 1-2s。
4. 观察点云地图成形与回环对齐；记录回环触发时的现象。
5. 保存与导出：
    - RTAB-Map 数据库：`ros2 service call /rtabmap/save ...`（或 Ctrl+C 自动存）

- 导出 OctoMap：`ros2 launch octomap_server octomap_mapping.launch.xml`（通用）

6. **无相机条件**：观看课程演示视频/截图，按上述流程在报告里写出完整操作步骤即可。

**易错点**：

- ⚠️ 相机与 RTAB-Map 话题名不匹配 → **解决**：用 `--ros-args -p rgb_topic:=... -p depth_topic:=...` 显式指定。
- ⚠️ 走太快三维地图"花" → **解决**：三维 SLAM 比二维更怕运动畸变，速度比实验 2 更慢。
- ⚠️ 回环触发时地图"乱一下" → **解决**：正常！这是图优化一次性校正所有帧的瞬间。

---

## 步骤4：三维导航概念（90min）

**知识点讲解**：

**三维导航（7.3.2）**

- **原理**：三维导航 = 用三维地图（OctoMap）替代二维栅格作为代价地图输入，让 Nav2 在**三维代价空间**中规划。好处：
  - 悬空障碍（悬空杆、桌面下沿）可被正确区分——低矮障碍可跨过/绕行，而非一刀切
  - 支持"高度分层"规划（如机械臂工作区与底盘通行区分离）
- **工程现实**：ROS2 生态中三维导航多用于特种场景（无人机、仓储机器人）；地面小车通常**二维就够**。三维地图更多服务于**感知/抓取/展示**。
- **白话类比**：二维地图像"楼层平面图"（只说哪能走），三维地图像"建筑信息模型 BIM"（每根管道每个柜子都建模）——建 BIM 贵，但碰撞检测准。

**操作步骤**：

1. 阅读 Nav2 的 costmap 如何消费地图话题（global_costmap static_layer 订阅 /map；三维场景可换 octomap 话题）。
2. 讨论（记录）：若把 OctoMap 二维投影后交给 Nav2，相当于"把三维信息降维成二维代价"，有什么损失与好处?
3. 选做（有环境条件）：把 octomap_server 输出的占据栅格话题接到 Nav2 global_costmap，试跑一次导航。

**易错点**：

- ⚠️ 以为三维导航 = 直接替换就能用 → **解决**：需要把 OctoMap 投影/转换成交给 Nav2 的格式，工程上有额外环节。
- ⚠️ 高估三维 SLAM 的实时性 → **解决**：RK3576 跑实时三维 SLAM 压力大；实时性优先场景建议二维。

---

## 步骤5：二维 vs 三维选型分析（60min）

**知识点讲解**：

**选型对比（7.4.1）**

- **原理**：从六个维度对比：

| 维度 | 二维 SLAM（slam_toolbox） | 三维 SLAM（RTAB-Map） |
| --- | --- | --- |
| 传感器 | 激光雷达 | RGB-D 深度相机 |
| 地图 | 栅格（俯视） | 点云/OctoMap（立体） |
| 算力 | 低（RK3576 轻松） | 高（实时需更强算力） |
| 精度 | 水平精度高 | 含高度信息，垂直精度依赖深度 |
| 导航 | Nav2 直接消费 | 需转换后消费 |
| 适用 | 地面巡检/送货 | 分层避障/抓取定位/展示 |

- **白话类比**：二维 = "平面图"（便宜够用）；三维 = "3D 模型"（信息全但贵）。选型 = 看任务需不需要"高度"这一维。

**操作步骤**：

1. 填写二维 vs 三维对比表（结合本课程 7 个实验）。
2. 场景判断题（记录结论并说明理由）：
    - A. 扫地机器人（家庭平面）→ 选二维
    - B. 仓库货架间多层取货 → 选三维（需要高度）
    - C. 无人机绕楼巡检 → 选三维
    - D. 走廊送餐小车 → 选二维
3. 写一段 200 字选型建议：什么信号提示你"该上三维了"？（提示：出现悬空障碍/高度分层/抓取定位需求时）

**易错点**：

- ⚠️ 跟风上三维 → **解决**：三维的算力与工程成本翻倍；地面任务优先二维，确有高度需求再上三维。
- ⚠️ 忽略深度相机局限性 → **解决**：RGB-D 在强光/远距/反光处失效，户外或大场景优先激光。

## 7.7 注意事项

- ⚠️ 本实验为原理+演示型（源码暂缺），**不强行要求跑通三维导航**；重点在理解与选型分析。
- ⚠️ 有 HP60C 相机时注意：深度相机有效距离 0.5-3m，避免直射强光。
- ⚠️ 三维建图对算力要求高，遥控速度比二维建图更慢（角速度 ≤0.5rad/s）。
- ⚠️ 三维 SLAM 地图文件大，保存后注意磁盘空间。

## 7.8 实验总结

**完成成果**：

- 理解 RTAB-Map 原理（图像特征、词袋回环、图优化）
- 理解 RGB-D 相机反投影与点云/OctoMap 表示
- 掌握三维建图的通用流程（相机→RTAB-Map→操控→保存）
- 完成二维 vs 三维 SLAM 选型分析与场景判断

**技能收获**：

- 技能1：RTAB-Map 通用启动与话题配置
- 技能2：深度图像与点云可视化观察
- 技能3：SLAM 技术选型分析（二维/三维）

**常见问题与解决**：

| 问题 | 原因 | 解决方法 |
| --- | --- | --- |
| 深度图全黑 | 超出有效距离/反光 | 保持 0.5-3m；避免强光 |
| 话题名不匹配 | 相机与 RTAB-Map 默认名不同 | -p rgb_topic/depth_topic 指定 |
| 三维地图花 | 运动太快 | 更慢速；转角停留 |
| 算力不足卡顿 | 三维实时 SLAM 开销大 | 降分辨率；关闭额外显示 |
| 回环乱跳 | 图优化一次性校正 | 属正常，记录即可 |

## 7.9 实验报告要求

**思考题**：

1. RTAB-Map 用什么手段做回环检测？与 slam_toolbox 的扫描匹配有何本质区别？
2. 写出 RGB-D 相机反投影公式并解释每个符号；为什么需要相机内参？
3. 点云地图与 OctoMap 各自的优缺点？导航为什么更常用 OctoMap？
4. 结合二维 vs 三维对比表，说明"什么场景必须上三维 SLAM"？并解释三维导航的工程难点。
5. 你认为三维 SLAM 在具身智能（如机械臂抓取、人机协作）中的价值是什么？

**评分标准**：RTAB-Map 原理 20 分 + RGB-D 与三维地图 20 分 + 三维建图流程 25 分 + 三维导航概念 15 分 + 选型分析 20 分 = 100 分。